



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИУ «Информатика и системы управления»

КАФЕДРА ИУ-7 «Программное обеспечение ЭВМ и информационные технологии»

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
К КУРСОВОЙ РАБОТЕ
НА ТЕМУ:

«Графический редактор многогранников»

Студент ИУ7-56Б
(Группа)

(Подпись, дата)

Мансуров В. М.
(Фамилия И.О.)

Руководитель курсовой работы

(Подпись, дата)

Майков К. А.
(Фамилия И.О.)

2022 г.

РЕФЕРАТ

Расчетно-пояснительная записка 69 с., 27 рис., 15 лист., 14 ист., 1 прил.

КОМПЬЮТЕРНАЯ ГРАФИКА, АЛГОРИТМЫ УДАЛЕНИЯ НЕВИДИМЫХ ЛИНИЙ, Z-БУФЕР, ЗАКРАСКА, ОСВЕЩЕНИЕ, СЦЕНА, ПОСТРОЕНИЕ ТЕНЕЙ.

Целью работы является разработка программы, позволяющее создавать модели многогранника с различными параметрами, изменение параметров модели.

Для визуализации сцены использовался алгоритм с Z-буфером, а для представления модели многогранника использовалась поверхностная модель, представленная в виде списка ребер.

В процессе работы были проанализированы различные алгоритмы, методы представления, закраски геометрических моделей на сцена. Выбраны наиболее подходящие технологии решения для поставленной задачи, а также разработаны алгоритмы для их программной реализации. Разработана программа, предназначенная для создания модели многогранника с различными геометрическими и спектральными параметрами.

Проведено исследование быстродействия программы при различном количестве создания объектов на сцене с фиксированным количеством ребер. Также с фиксированным количеством объектов, но с различным количеством боковых ребер. Из результатов исследования следует, что время отрисовки сцены увеличивается как при увеличении количества объектов на сцене, так и при увеличении количества боковых ребер.

СОДЕРЖАНИЕ

РЕФЕРАТ	2
ОПРЕДЕЛЕНИЯ	3
ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ	4
ВВЕДЕНИЕ	5
1 Аналитическая часть	6
1.1 Формализация объектов синтезируемой сцены	6
1.2 Способы описания трехмерных геометрических моделей на сцене	7
1.3 Способы задания поверхностных моделей	8
1.4 Анализ алгоритмов удаления невидимых линий и поверхностей	12
1.4.1 Алгоритм Робертса	12
1.4.2 Алгоритм Варнока	14
1.4.3 Алгоритм, использующий Z-буфер	15
1.4.4 Алгоритм прямой трассировки лучей	16
1.4.5 Алгоритм обратной трассировки лучей	17
1.4.6 Выбор оптимального алгоритма	18
1.5 Анализ алгоритмов закраски	19
1.5.1 Простая закраска	20
1.5.2 Закраска по Гуро	20
1.5.3 Закраска по Фонгу	22
1.5.4 Выбор алгоритма закраски	22
1.6 Анализ алгоритма построения теней	23
1.7 Анализ моделей освещения	24
1.7.1 Модель Ламберта	25
1.7.2 Модель Фонга	26

2	Конструкторская часть	29
2.1	Требования к программному обеспечению	29
2.2	Описание структур данных	29
2.3	Общий алгоритм построения изображения	30
2.4	Аффинные преобразования	32
2.5	Приведение к пространству камеры	33
2.6	Перспективная проекция	34
2.7	Невидимые грани	35
2.8	Пирамида видимости	36
2.9	Алгоритм Z-буфера	38
3	Технологическая часть	41
3.1	Средства реализации	41
3.2	Структура программы	42
3.3	Реализация алгоритмов	44
3.4	Интерфейс программного обеспечения	58
4	Исследовательская часть	63
4.1	Постановка эксперимента	63
4.1.1	Технические характеристики	63
4.1.2	Результаты эксперимента	63
	ЗАКЛЮЧЕНИЕ	66
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	67
	ПРИЛОЖЕНИЕ А	69

ОПРЕДЕЛЕНИЯ

Тангаж — угловое движение объекта, при котором его продольная ось изменяет свое направление относительно горизонтальной плоскости [1].

Рыскание — угловые движения объекта вокруг вертикальной оси [1].

Graphical user interface — «Графический интерфейс пользователя» обеспечивает возможность управления поведением вычислительной системы через визуальные элементы управления — окна, списки, кнопки, гиперссылки и т.д. [2]

Буфер кадра — часть графической памяти для хранения массива кодов, определяющих засветку пикселей на экране [2].

Визуализация (англ. Rendering) — создание плоских изображений трехмерных (3D) моделей [2].

Перспективная (центральная) проекция — вид проекции, где лучи проектирования исходят из одного центра (центра проектирования), размещенного на конечном расстоянии от объектов и плоскости проектирования [2].

ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

В текущей расчетно-пояснительной записке применяются следующие сокращения и обозначения.

ПО — программное обеспечение.

GUI — «Графический интерфейс пользователя».

ВВЕДЕНИЕ

Компьютерная графика — собой совокупность методов и способов преобразования информации в графическое представление при помощи ЭВМ. Данная область предоставляет широкий спектр алгоритмов: от воссоздания художественных эффектов в компьютерных играх до построения трехмерных объектов при моделировании сложных технологических продуктов. Но алгоритмы компьютерной графики ресурсозатратны — чем более качественное изображение требуется получить, тем больше накладных ресурсов (времени и памяти) тратится на его синтез. Это самая главная проблема при создании реалистичных изображений и динамических сцен.

Целью данного курсового проекта является разработка редактора композиции, состоящих из многогранников, геометрические и спектральные характеристики которые задаются пользователем. Для формирования более полного представления о полученной сцене должны присутствовать возможности передвижение камеры и источника света, и изменение спектральных характеристик источника света.

Для достижения поставленной цели необходимо решить следующие задачи:

- анализ существующих алгоритмов компьютерной графики, использующие для создание реалистичной модели и трехмерной сцены;
- выбрать наиболее подходящих алгоритмов для решения поставленной задачи;
- проектирование архитектуры и графического интерфейса программы;
- выбрать средства реализации программного обеспечения;
- разработка ПО и реализация выбранных алгоритмов и структур данных;
- провести замеры временных характеристик разработанного программного обеспечения.

1 Аналитическая часть

В данной части проводится анализ объектов сцены и существующих алгоритмов построения изображений и выбор более подходящих алгоритмов для дальнейшего использования.

1.1 Формализация объектов синтезируемой сцены

Сцена состоит из определенного набора объектов.

- 1) Направленный источник света представляет собой вектор направления света и принимает ортогональную проекцию визуализируемой сцены из своего положения с некоторым ограниченным положением. В зависимости от расположения источника и направления распространения лучей света, определяется тень от объекта, расположенных на сцене. Цвет свечения описывается через RGB параметры.
- 2) Объекты сцены – примитивные многогранники, расположенные в пространстве сцены. Каждая модель многогранника представляет собой набор граней, описываемых точками в пространстве, которые соединены ребрами и задается параметрическими (радиус нижнего и верхнего основания, количество боковых граней и высота) и спектральными характеристиками (зеркальный коэффициент, коэффициент блеска и цвет) для построения.
- 3) Камера характеризуется своим пространственным положением и направлением просмотра.

1.2 Способы описания трехмерных геометрических моделей на сцене

В компьютерной графике для описания трехмерных геометрических объектов существует три типа моделей: каркасная, поверхностная и твердотельная [3]. Использование моделей позволяет правильно отображать форму и размеры объектов сцены.

- *Каркасная модель* — простейший вид моделей, который содержит минимум информации только о вершинах и ребрах объектов. Это моделирование самого низкого уровня, которое имеет ряд серьезных ограничений, большинство из которых возникает из-за недостатка информации о гранях, которые заключены между ребрами. Невозможно выделить внутреннюю и внешнюю область изображения твердого объемного тела. Однако каркасная модель требует меньше памяти и затрат времени на построение, что достаточно пригодна для решения задач, не требующие информации о поверхности объекта (например, если в объекте есть отверстия). Проблемой этой модели заключается в том, что она не позволяет отличить видимые грани от невидимых. Операции по удалению невидимых линий можно выполнить только вручную с применением команд редактирования каждой отдельной линии, но результат работы нарушает каркасную конструкцию, по причине того что линии невидимы в одном случае и видимы в другом. Кроме того, каркасная модель не несет информации о поверхностях, ограничивающих форму, что обуславливает невозможность обнаружения нежелательных взаимодействий между гранями объекта [4].
- *Поверхностная модель* часто используется в компьютерной графике, кроме содержания информации о вершинах и ребрах, содержит еще информацию о поверхности. При построении поверхностной модели пред-

полагается, что технические объекты ограничены поверхностями, которые отделяют их от окружающей среды. Недостатком поверхностной модели является отсутствие информации о том, с какой стороны поверхности находится материал.

- *Твердотельная модель* отличается от поверхностной тем, что в данной модели к информации о поверхностях добавляется информация о том, с какой стороны расположен материал. Это достигается путем указания направления внутренней нормали.

Для решения поставленной задачи не подойдет каркасная модель, так как такое представление будет приводить к неправильному восприятию форм объекта. Твердотельная модель также не подойдет, так как по поставленной задаче нет необходимости знать из какого материала будет выполнен тот или иной объект и с какой стороны расположен материал. Поэтому выбор остается лишь поверхностной модели.

1.3 Способы задания поверхностных моделей

Поверхностная модель задается следующими способами [4, ?].

- *Аналитический (параметрический) способ* характеризуется описанием модели объекта, которое доступно в неявной форме, то есть для получения визуальных характеристик необходимо дополнительно вычислять некоторую функцию, которая зависит от параметра, что в свою очередь довольно удобно при просчете поверхностей вращения.
- *Полигональная сетка* характеризуется совокупностью вершин, ребер и граней, определяющих форму объекта в трехмерном пространстве.

Рассмотрим существующие способы хранения информации о полигональной сетке.

Вершинное представление описывает объект множество вершин, соеди-

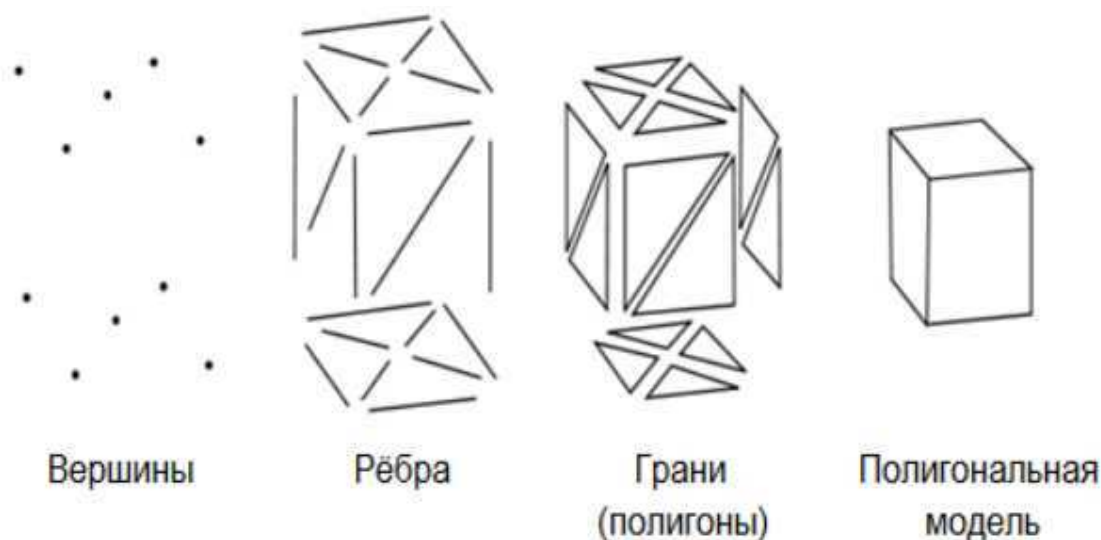


Рисунок 1 – Полигональная сетка

ненных с другими вершинами (вершины, которые указывают на другие вершины). Информация о ребрах и гранях неявно присутствует в представлении из-за чего для восстановления исходного тела необходимо обойти все вершины и составить списки граней. Кроме того, операции с ребрами и гранями выполнить нелегко [3]. Тем не менее из-за простоты представления дает возможность множество операций над сеткой. Хранение информации о сетке требует не так много памяти по сравнению с другими способами. На рисунке 2 представлен пример вершинного представления, рассмотренный на кубе.

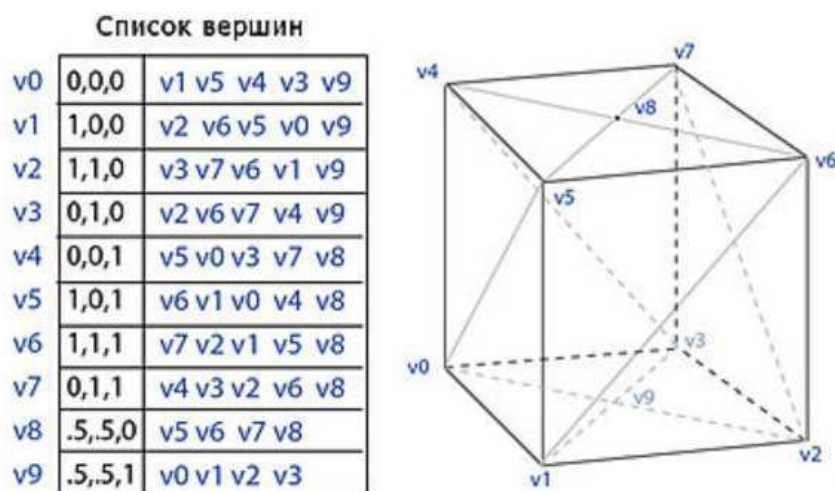


Рисунок 2 – Вершинное представление

Представление называемый списком граней представляет объект не только множеством вершин, но граней. В отличие от предыдущего способа, вершины и грани определены явно, благодаря чему нахождение соседних вершин и граней довольно проста и поиск соседних граней и вершин занимает постоянное время [3]. Также список вершин содержит список граней, связанных с каждой вершиной. Однако ребра неявны, поэтому поиск все равно необходим, чтобы найти все грани, окружающие данную грань. Другие динамические операции, такие как разделение или слияние граней, также сложны с сетками граней и вершин. Пример представления список граней представлен на рисунке 3.

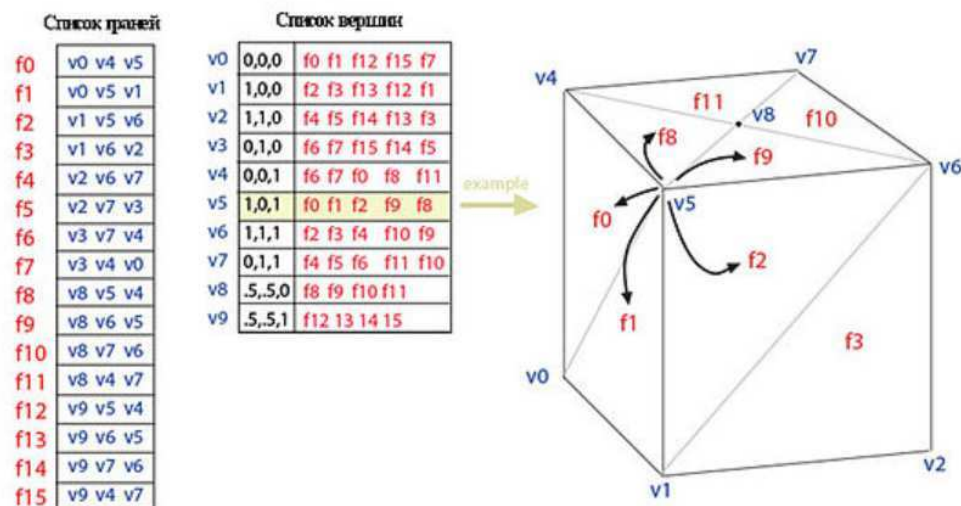


Рисунок 3 – Список граней

Представление называемый таблицей углов представляет специальной таблицей, хранящей вершины, обход которых неявно задает полигоны. Такое представление более компактно и более производительнее для нахождения полигонов, но в связи с тем, что вершины присутствуют в описании нескольких углов, операции по их изменению медленны. Также угловые таблицы представляют сетки не полностью, из-за чего их требуется несколько.

«Крылатое» представление задает точки, каждая из которых указывает на две вершины, две грани и четыре ребра, которые ее касаются, благодаря

чему обход поверхностей выполняется за постоянное время. Однако метод очень требует памяти и изменения геометрических характеристик приводит к формированию списка индексов граней. Данный способ полезен для определения столкновений объектов. Пример представлен на рисунке 4.

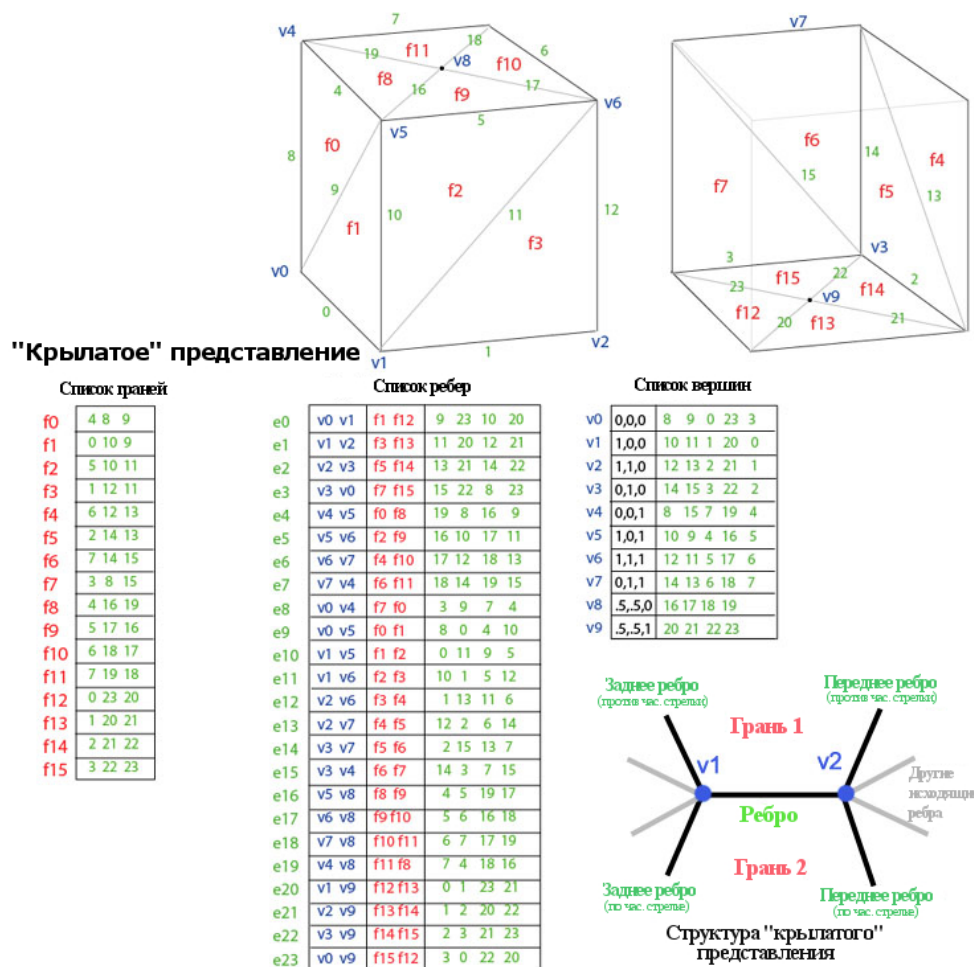


Рисунок 4 – «Крылатое» представление

Оптимальное для реализации представление многогранника — модель, заданная полигональной сеткой. Такая модель позволит избежать проблем при описании сложных объектов сцены. Способом хранения полигональной сетки был выбран список граней, так как это даст явное описание граней. Этот способ позволит эффективно преобразовывать модели, так как структура будет включать в себя список вершин.

1.4 Анализ алгоритмов удаления невидимых линий и поверхностей

Основной задачей при построении реалистичного изображения является задача удаления объектов или их частей, которые перекрываются другими объектами, то есть являются невидимыми с точки зрения наблюдателя. Выделяют две группы алгоритмов для ее решения:

- Алгоритмы, работающие в объектном пространстве. Данные алгоритмы имеют привязку к мировой или физической системе координат. Получаемые результаты ограничиваются только точностью вычислений, однако требуют большого объема вычислений, зависящего от требуемой точности и сложности поступающей на вход сцены. В эту группу входят алгоритм Робертса, алгоритм со списком приоритетов и т.д. [4]
- Алгоритмы, работающие в пространстве изображения. Данные алгоритмы предполагают привязку к системе координат экрана или картинной плоскости, на которую производится проецирование изображаемых объектов. Объем требуемых вычислений значительно меньше, чем у алгоритмов первой группы, и зависит от разрешающей способности экрана и количества объектов на сцене. Основными представителями данной группы являются алгоритм Варнака, алгоритм Z-буфера и алгоритм трассировки лучей [4].

1.4.1 Алгоритм Робертса

Данный алгоритм работает в объектном пространстве, решая задачу только с выпуклыми телами, и выполняется в 3 этапа [4].

Первый этап — подготовка исходной матрицы V , которая задает инфор-

мацию о каждой фигуре. Размерность матрицы — $4 * n$, где n — количество граней тела. Каждый столбец матрицы представляет собой четыре коэффициента уравнения плоскости $ax + by + cz + d = 0$, проходящей через очередную грань. Таким образом, матрица тела будет представлена в следующем виде:

$$V = \begin{pmatrix} a_1 & a_2 & \dots & a_n \\ b_1 & b_2 & \dots & b_n \\ c_1 & c_2 & \dots & c_n \\ d_1 & d_2 & \dots & d_n \end{pmatrix} \quad (1)$$

Матрица тела должна быть сформирована корректно, то есть любая точка, расположенная внутри тела, должна располагаться по положительную сторону от каждой грани тела. В случае, если для очередной грани условие не выполняется, соответствующий столбец матрицы надо умножить на -1 . Для проведения проверки следует взять точку, расположенную внутри тела. Координаты такой точки можно получить путем усреднения координат всех вершин тела.

Второй этап — удаления ребер, экранируемых самим телом, где рассматривается вектор взгляда $E = \{0, 0, -1, 0\}$. Для определения невидимых граней достаточно умножить вектор E на матрицу тела V . Отрицательные компоненты полученного вектора будут соответствовать невидимым граням.

Третий этап — удаление невидимых ребер, экранируемых другими телами сцены. Для определения невидимых точек ребра требуется построить луч, соединяющий точку наблюдения с точкой на ребре. Точка будет невидимой, если луч на своем пути встречает в качестве преграды рассматриваемое тело. Если тело является преградой, то луч должен пройти через тело. Если луч проходит через тело, то он находится по положительную сторону от каждой грани тела.

В данном алгоритме главным недостатком является его вычислитель-

ная трудоемкость равная $O(n^2)$, где n — количество объектов на сцене. Также все тела на сцене должны быть выпуклыми, что приводит к дополнительным проверкам. Однако, работа в объектном пространстве и высокая сложность вычислений обеспечивает высокую точность результата.

1.4.2 Алгоритм Варнока

Алгоритм Варнока работает в пространстве изображения и позволяет определить, какие грани или части граней объектов сцены видимы, а какие заслонены другими объектами [4, 5]. Алгоритм предлагает разбиение области изображения на более мелкие окна, и для каждого такого окна определяются связанные с ней многоугольники и те, видимость которых можно определить, изображаются на сцене. На рисунке 5 представлен пример разбиения с помощью алгоритма Варнака.

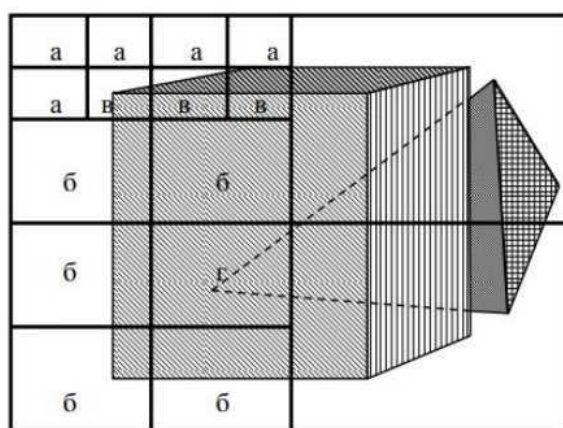


Рисунок 5 – Пример разбиения алгоритмом Варнака

В качестве граней обычно выступают выпуклые многоугольники, алгоритмы работы с ними эффективнее, чем с произвольными многоугольниками. Окно, в котором необходимо отобразить сцену, должно быть прямоугольным. Алгоритм работает рекурсивно, что является его главным недостатком, на каждом шаге анализируется видимость граней и, если нельзя «легко» опре-

делить видимость, окно делится на 4 части и анализ повторяется отдельно для каждой из частей.

1.4.3 Алгоритм, использующий Z-буфер

Алгоритм, использующий z-буфер, является одним из самых простых и широко используемых и работает в пространстве изображения. Идея z-буфер является простым обобщением идеи о буфере кадра [4]. Используется два буфера:

- буфер кадра, который используется для запоминания атрибутов (интенсивности) каждого пиксела;
- z-буфер — отдельный буфер глубины, используемый для запоминания координаты z каждого пиксела.

Первоначально в z-буфере находятся минимально возможные значения z, а в буфере кадра располагаются пиксели, описывающие фон. В процессе работы глубина z каждого нового пиксела, который нужно занести в буфер кадра, сравнивается с глубиной того пиксела, который уже занесен в z-буфер. Если новый пиксель расположен ближе к наблюдателю, чем предыдущий, то он заносится в буфер кадра и происходит корректировка z-буфера. Затем каждый многоугольник преобразуется в растровую форму и записывается в буфер кадра, при этом не производится начального упорядочения. В противном случае никаких действий не производится [4]. На рисунке 6 представлена пример работы алгоритма Z-буфера.

Основным преимуществом алгоритма является простота реализации. Трудоемкость алгоритма увеличивается линейно в зависимости от количества объектов на сцене. Особая эффективность по времени достигается за счет того, что нет необходимости в сортировке объектов. Это является большим плюсом, так как на сцене может быть большое количество объектов. Однако

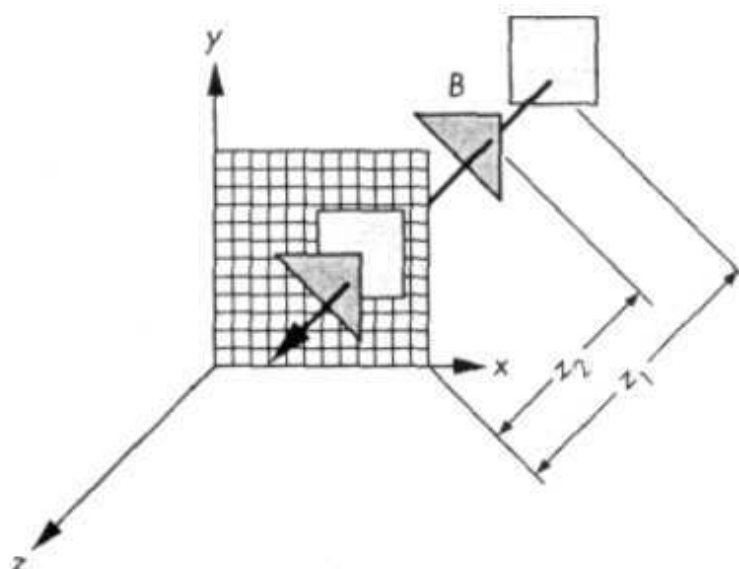


Рисунок 6 – Алгоритм с Z-буфером

алгоритм очень требователен к памяти из-за необходимости хранить информацию о каждом пикселе и выделение памяти под два буфера. Реализация эффектов прозрачности и устранения лестничного эффекта осложнена.

1.4.4 Алгоритм прямой трассировки лучей

Главная идея, лежащая в основе алгоритма трассировки лучей, заключается в том, что наблюдатель видит любой объект посредством испускаемого неким источником света, который падает на этот объект и затем согласно законам оптики некоторым путем доходит до глаза наблюдателя. В методе прямой трассировки предполагает построения траекторий лучей от всех источников света ко всем точкам всех объектов сцены, отражаются и преломляются или проходят сквозь него и в результате достигает наблюдателя. Данные лучи называются первичными. Если объект не является отражающим или прозрачным, то траектория луча на этой точке обрывается [4, 6].

Основным недостатком алгоритма является излишне большое число рассматриваемых лучей, приводящее к существенным затратам вычислительных мощностей, так как лишь малая часть лучей достигает точки наблюде-

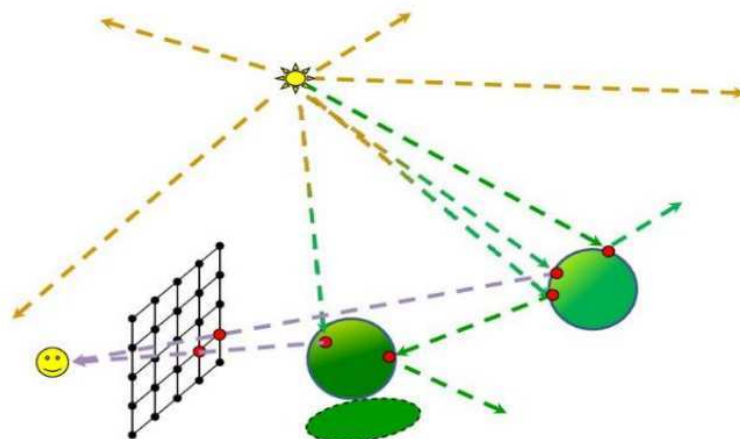


Рисунок 7 – Алгоритм прямой трассировки лучей

ния. Данный алгоритм подходит для генерации статических сцен и моделирования зеркального отражения, а так же других оптических эффектов как отражение, преломление и т.д.

1.4.5 Алгоритм обратной трассировки лучей

Наблюдатель видит объект посредством испускаемого источником света, который падает на этот объект и согласно законам оптики некоторым путем доходит до глаза наблюдателя. Отслеживать пути лучей от источника к наблюдателю неэффективно с точки зрения вычислений, поэтому наилучшим способом будет отслеживание путей в обратном направлении, то есть от наблюдателя к объекту. Лучи испускаются из камеры, пронизывая каждый пиксель сцены. После этого определяют пересечение первичного луча с объектами сцены. Первичный луч – луч, выпущенный из камеры. Если оно было установлено, вычисляют интенсивность пиксела, учитывая положения источников света [4, 6].

Считается, что наблюдатель расположен на положительной полуоси z в бесконечности, поэтому все световые лучи параллельны оси z . В ходе работы испускаются лучи от наблюдателя и ищутся пересечения луча и всех объек-

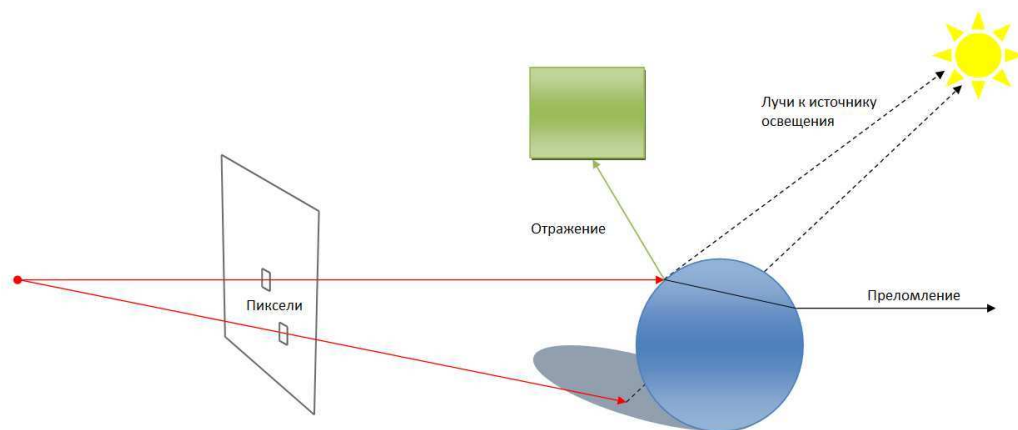


Рисунок 8 – Алгоритм обратной трассировки лучей

тов сцены. В результате пересечение с максимальным значением z является видимой частью поверхности и атрибуты данного объекта используются для определения характеристик пиксела, через центр которого проходит данный световой луч. Для расчета эффектов освещения сцены проводятся вторичные лучи от точек пересечения ко всем источникам света. Если на пути этих лучей встречается непрозрачное тело, значит, данная точка находится в тени.

Несмотря на более высокую эффективность алгоритма в сравнении с прямой трассировкой лучей, данный алгоритм считается достаточно медленным, так как в нем происходит точный расчет сложных аналитических выражений для нахождения пересечения с рассматриваемыми объектами [5].

1.4.6 Выбор оптимального алгоритма

При реализации редактора композиций из графических примитивов многогранников необходимо обеспечить плавную смену кадров при перемещении камеры и изменении объектов, поэтому алгоритм должен иметь минимальную зависимость трудоемкости алгоритма от числа объектов на сцене и использование рекурсивных вызовов. Также требуется учесть ограничение разработки курсового проекта, поэтому требуется простота реализации ал-

горитма. Задействования дополнительной памяти под буферы не является весомым критерием и может быть опущен.

Таким образом, подходящий алгоритм является алгоритм с Z-буфером, который подходит для решения создания сцены с различным количеством объектов, что позволит иметь динамическую сцену. Данный алгоритм имеет линейную зависимость от числа объектов, что приведет к оптимальной работе программы. Также он простой в реализации и используется в большинстве графических движков, что приведет к быстрой скорости реализации алгоритма.

1.5 Анализ алгоритмов закраски

Методы закраски используются для затенения полигонов модели в условиях некоторой сцены, имеющей источники освещения. Учитывая взаимное положение рассматриваемого полигона и источника света, можно найти уровень освещенности по закону Ламберта [4]:

$$I_{\alpha} = I_0 \cdot \cos(\alpha), \quad (2)$$

где I_{α} - уровень освещенности в рассматриваемой точке, I_0 - максимальный уровень освещенности, а α - угол между вектором нормали к плоскости и вектором, направленным от рассматриваемой точки к источнику освещения.

Существует три основных алгоритма, позволяющих закрасить полигональную модель.

1.5.1 Простая закраска

Вся грань закрашивается одним уровнем интенсивности, который высчитывается по закону Ламберта. При данной закраске все плоскости (в том числе и те, что аппроксимируют фигуры вращения), будут закрашены одинаково, что в случае с фигурами вращения будет давать ложные ребра [4].

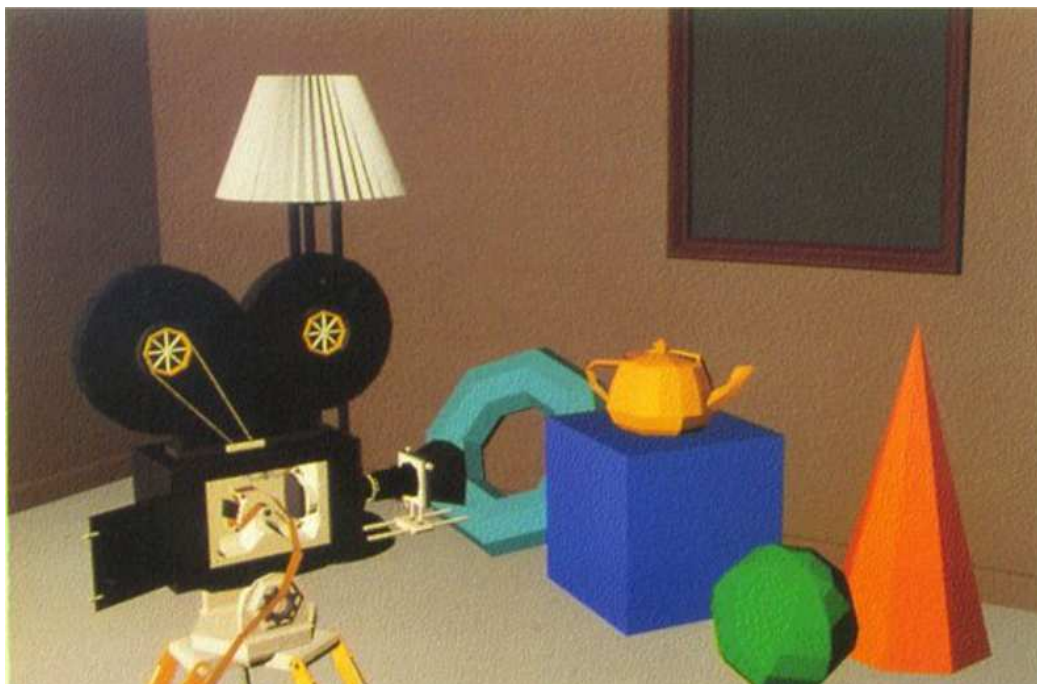


Рисунок 9 – Результат работы алгоритма простой закраски

Данный метод закраски обладает большим быстродействием, однако все пиксели грани имеют одинаковую интенсивность, и сцена выглядит нереалистично. Тем не менее, этот метод крайне прост в реализации и совершенно не требователен к ресурсам.

1.5.2 Закраска по Гуро

Метод Гуро устранить дискретность изменения интенсивности и создать иллюзию гладкой криволинейной поверхности. Он основан на интерполяции интенсивности.

Данный метод отличается от простой закраски тем, что разные точки грани закрашиваются с разными значениями интенсивности. Для это в каждой вершине грани находится вектор нормали и вычисляется значение интенсивности. Затем найденные значения интенсивности интерполируются по всем точкам примыкающих граней. На рисунке 10 представлен пример работы алгоритма закраски по Гуро.

Закрашивание граней по методу Гуро осуществляется в четыре этапа.

- 1) Вычисляются нормали к каждой грани.
- 2) Определяются нормали в вершинах. Нормаль в вершине определяется усреднением нормалей примыкающих граней.
- 3) На основе нормалей в вершинах вычисляются значения интенсивностей в вершинах согласно выбранной модели отражения света.
- 4) Закрашиваются полигоны граней цветом, соответствующим линейной интерполяции значений интенсивности в вершинах.



Рисунок 10 – Результат работы алгоритма закраски по Гуро

Метод Гуро применим для небольших граней, расположенных на значительном расстоянии от источника света. Если же размер грани большой, то расстояние от источника света до центра будет меньше, чем до вершин,

и центр грани должен выглядеть ярче, чем ребра. Но из-за линейного закона, используемого в интерполяции, метод не позволяет это сделать, поэтому появляются участки с неестественной освещенностью

1.5.3 Закраска по Фонгу

Закраска Фонга по своей идее похожа на закраску Гуро, но ее отличие состоит в том, что в методе Гуро по всем точкам полигона интерполируется значения интенсивностей, а в методе Фонга – вектора нормалей, и с их помощью для каждой точки находится значение интенсивности [4]. На рисунке 11 представлен пример работы алгоритма закраски по Фонгу.

Этапы следующие.

- 1) Определяются нормали к граням.
- 2) По нормальям к граням определяются нормали в вершинах. В каждой точке закрашиваемой грани определяется интерполированный вектор нормали.
- 3) По направлению векторов нормали определяется цвет точек грани в соответствии с выбранной моделью отражения света.

Эта закраска требует больших вычислительных затрат, чем предыдущие, однако она позволяет достигнуть лучшей локальной аппроксимации кривизны поверхности и, следовательно, с ее помощью получается более реалистичное изображение.

1.5.4 Выбор алгоритма закраски

Алгоритм закраски Фонга требует большего числа вычислений по сравнению с другими, однако он дает наиболее реалистичное изображение, в част-



Рисунок 11 – Результат работы алгоритма закраски по Фонгу

ности зеркальных бликов. В данном курсовом проекте пользователь должен иметь возможность задавать спектральные характеристики объектов, в том числе коэффициенты зеркального отражения и блеска поверхности, поэтому для получения более реалистичной сцены используется метод закраски Фонга.

1.6 Анализ алгоритма построения теней

Для генерации теней в данном курсовом проекте используется метод теневых карт. Он основан на алгоритме Z-буфера: сначала камера устанавливается на позицию источника освещения и происходит построение сцены с занесением глубин в теневой буфер (карту теней). Все точки в этом буфере видимы с точки зрения источника, соответственно они освещены. После этого изображение строится уже с позиции наблюдателя, при этом для каждой точки буфера кадра расстояние до источника освещения сравнивается со значением из теневой карты: если это расстояние больше полученного значения

из карты, то точка лежит в тени и отображается только с учетом рассеянного света. Иначе точка освещена и происходит расчет ее интенсивности со всеми составляющими освещения.

Так как используемый источник освещения является точечным, то есть излучающим свет во всех направлениях, то для генерации теней необходимо произвести построение сцены 6 раз для разных направлений камеры: вдоль и против осей OX , OY и OZ . В результате образуется так называемая кубическая карта теней, в которой содержатся значения глубин всех освещенных участков сцены.

Этот способ генерации теней требует значительного числа вычислений, однако они выполняются только при перестроении сцены, то есть при изменении положения камеры заново вычислять значения в карте теней не нужно. Таким образом метод кубической карты теней позволяет сохранить плавность подачи картинки при перемещении по сцене, поэтому используется в данном курсовом проекте.

1.7 Анализ моделей освещения

Все модели освещения делятся на два вида: глобальные и локальные. Глобальные модели учитывают возможности отражения и преломления света от объектов, не являющихся прямыми источниками освещения, поэтому они требуют значительных затрат. Например, прозрачные поверхности должны позволять видеть объекты за ними, на поверхностях возможно появление таких явлений, как рефлексy. Глобальные модели освещения основываются на том, что видимость и затенения связаны между собой: яркость точки поверхности определяется распределением яркости по всем остальным поверхностям, видимым из этой точки. Так можно моделировать шероховатость и свойства отражения реальных материалов, освещение многократно отражен-

ным светом и связанные с ним цветовые эффекты [7].

Существуют более простые, локальные модели освещения, которые учитывают только свет от источника. Они вычисляют интенсивность, цвет и дальнейшее распределение света на поверхности, но не учитывают перенос света между объектами. Изображение формируется в результате отражения, падающего на поверхность объектов света, интенсивность и цвет которого необходимо рассчитать. Фоновое освещение определяет цвет объекта в тени или при отсутствии явных источников освещения. Его интенсивность постоянна, а также равномерно распределена по всему пространству.

Выделяют две основные модели локального освещения: модель Ламберта и модель Фонга.

1.7.1 Модель Ламберта

Модель Ламберта моделирует идеальное диффузное освещение. Считается, что свет при попадании на поверхность рассеивается равномерно во все стороны. На рисунке 12 показано, что согласно этой модели, освещенность в точке определяется только плотностью света в точке поверхности, а она линейно зависит от косинуса угла падения. При этом положение наблюдателя не имеет значения, т.к. диффузно отраженный свет рассеивается равномерно по всем направлениям [7].

Пусть:

- $\vec{L}$ — вектор от точки до источника;
- $\vec{N}$ — вектор нормали;
- I — результирующая интенсивность света в точке;
- I_0 — интенсивность источника;
- K_d — коэффициент диффузного освещения.

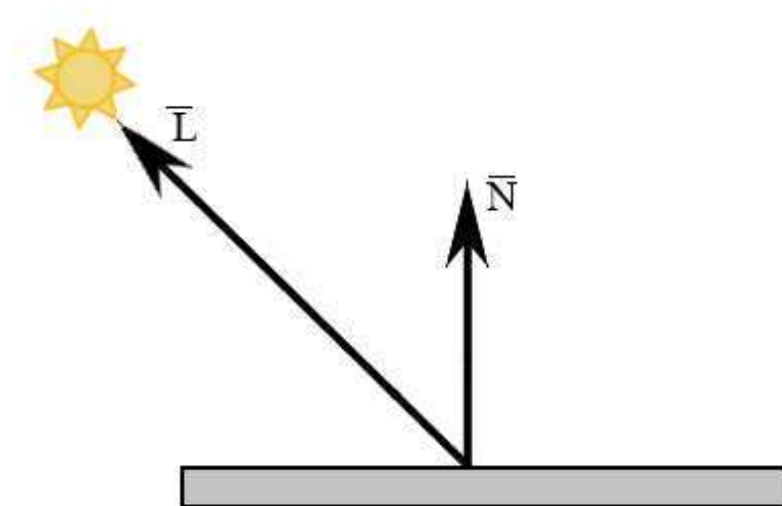


Рисунок 12 – Модель освещения Ламберта

Формула расчета интенсивности имеет следующий вид:

$$I = I_0 \cdot K_d \cdot \cos(\vec{L}, \vec{N}) = I_0 \cdot K_d \cdot (\vec{L}, \vec{N}) \quad (3)$$

Из формулы (5) следует главный недостаток модели Ламберта – одинаковая интенсивность во всех точках, принадлежащих одной грани.

Модель Ламберта является одной из самых простых моделей освещения. Данная модель очень часто используется в комбинации других моделей, практически в любой другой модели освещения можно выделить диффузную составляющую. Более-менее равномерная часть освещения (без присутствия какого-либо всплеска) как правило будет представляться моделью Ламберта с определенными характеристиками.

1.7.2 Модель Фонга

Модель Фонга – классическая модель освещения. Модель представляет собой комбинацию диффузной составляющей (модели Ламберта) и зеркальной составляющей и работает таким образом, что кроме равномерного освеще-

щения на материале может еще появляться блик. Падающий и отраженный лучи лежат в одной плоскости с нормалью к отражающей поверхности в точке падения, и эта нормаль делит угол между лучами на две равные части как показано на рисунке 13.

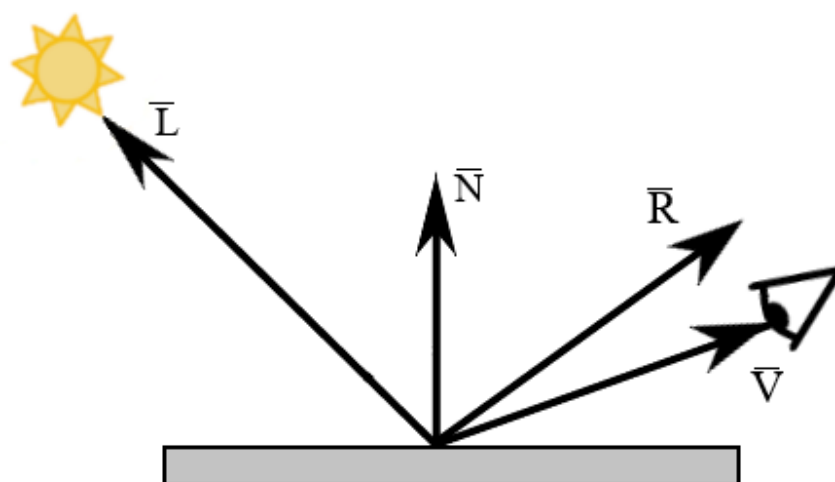


Рисунок 13 – Модель освещения Фонга

Отраженная составляющая освещенности в точке зависит от того, насколько близки направления на наблюдателя и отраженного луча. Также в модели освещения Фонга используется понятие рассеянного освещения – это константа, которая прибавляется к интенсивности в точке для придания сцене большей реалистичности.

Таким образом, согласно модели Фонга интенсивность к точке складывается будет рассчитываться по формуле:

$$I = I_a + I_d + I_s, \quad (4)$$

где I_a — диффузная составляющая, I_d — рассеянная составляющая, I_s — зеркальная составляющая.

Пусть:

- $\vec{R}$ — вектор отраженного луча;
- $\vec{V}$ — вектор от точки до наблюдателя;

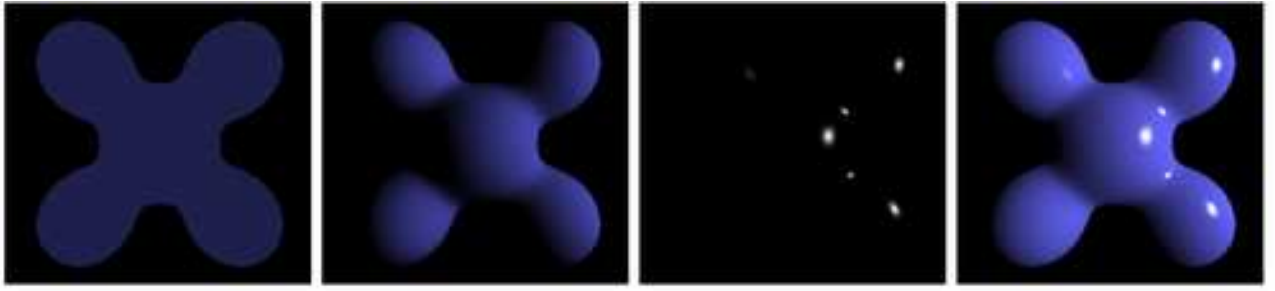


Рисунок 14 – Составляющие модели Фонга (слева направо: рассеянная, диффузная, зеркальная, суммарная)

- I_p — интенсивность рассеянного освещения;
- K_3 — коэффициент зеркального освещения;
- K_a — коэффициент рассеянного освещения;
- α — коэффициент блеска.

Формула для расчета интенсивности для модели Фонга имеет вид:

$$\begin{aligned}
 I &= I_p \cdot K_a + I_0 \cdot K_d \cdot \cos(\vec{L}, \vec{N}) + I_0 \cdot K_3 \cdot \cos^\alpha(\vec{R}, \vec{V}) = \\
 &= I_p \cdot K_a + I_0 \cdot K_d \cdot (\vec{L}, \vec{N}) + I_0 \cdot K_3 \cdot (\vec{R}, \vec{V})^\alpha
 \end{aligned}
 \tag{5}$$

Вывод

Подведя итог и проанализировав все вышеописанные алгоритмы методов освещения, можно сделать вывод, что наилучшим алгоритмом для решения задачи будет метод Фонга в сочетании с закраской Фонга, так как он позволяет получить реалистичную картинку с эффектами отражения и преломления лучей.

2 Конструкторская часть

В данном разделе представлены требования к программному обеспечению, рассмотрены алгоритмы, выбранные для построения сцены

2.1 Требования к программному обеспечению

Программа должна предоставлять графический интерфейс с функционалом:

- задать параметрические и спектральные характеристики, добавляемой модели многогранника;
- изменение положение модели многогранника в пространстве;
- изменение положение камеры и направления ее взгляда в пространстве;
- изменение характеристик модели многогранника.

Разработанное ПО должно соответствовать следующим требованиям:

- источник света создается при запуске программы;
- нельзя иметь больше 1 камеры в пространстве;
- программа должна корректно обрабатывать ввод некорректных данных;
- все объекты создаются путем ввода его параметрических и спектральных характеристик.

2.2 Описание структур данных

Для формирования общего алгоритма синтеза изображения в данной программе, необходимо ввести определения использующихся в ней структур данных.

- 1) Сцена представляет собой список с произвольным числом моделей, объ-

ект камеры и объект источника освещения.

2) Модель многогранника включает в себя следующие данные:

- массив вершин фигуры;
- массив полигонов фигуры;
- массив векторов нормалей к вершинам;
- коэффициенты отражения и блеска поверхности;
- цвет поверхности;
- матрица аффинных преобразований.

3) Камера содержит:

- положение в пространстве;
- значения углов тангажа и рыскания;;
- систему координат камеры, задаваемую тремя ортогональными векторами;
- угол обзора и соотношение сторон экрана;
- цвет поверхности;
- границы пирамиды видимости.

4) Источник освещения включает: положение в пространстве и цвет источника.

2.3 Общий алгоритм построения изображения

Алгоритм генерации изображения представлен на рисунке 15. На вход подается геометрические характеристики моделей, камеры, источника, освещения, спектральные характеристики моделей и источника освещения. На выход выдается результат построения буфера кадра сцены.

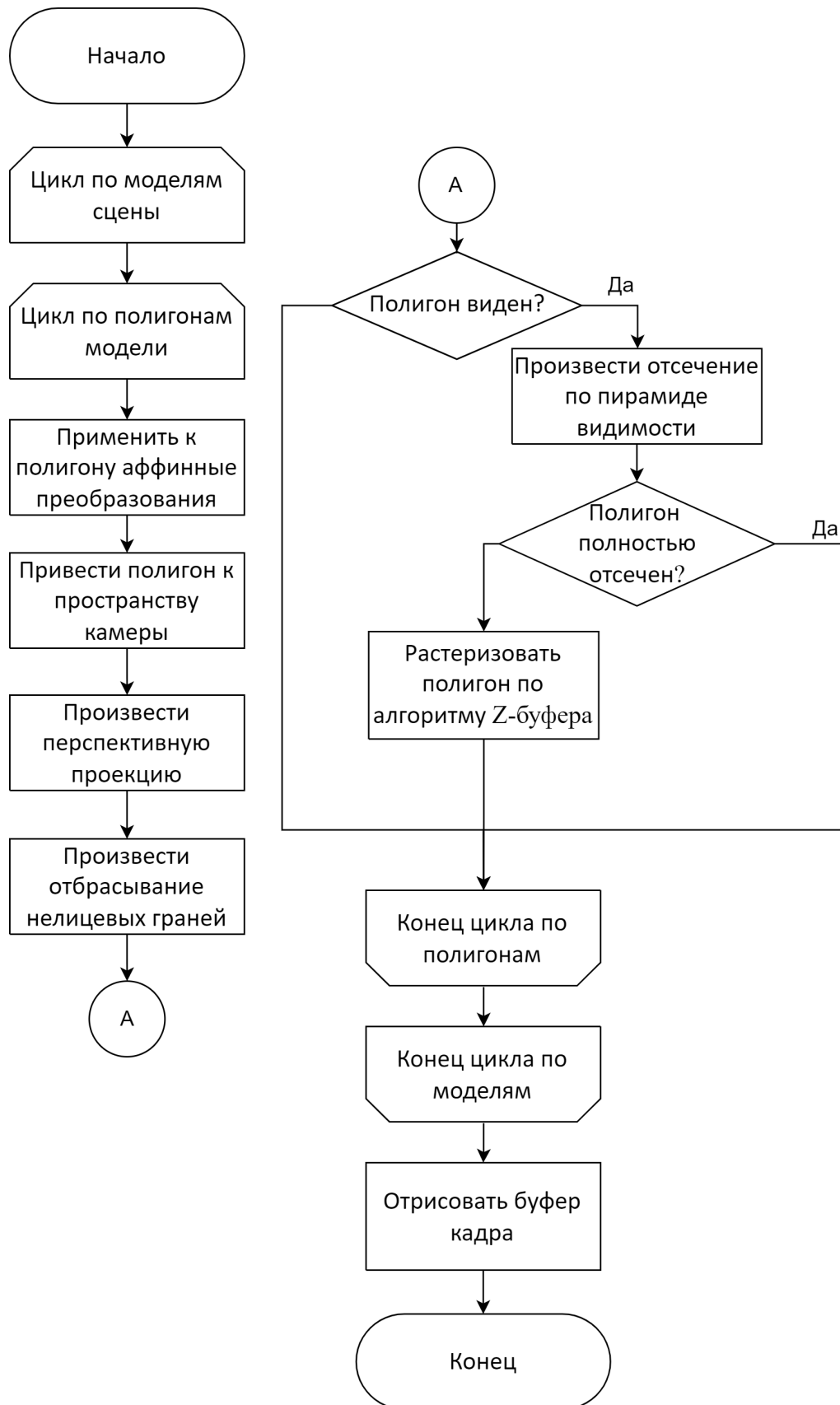


Рисунок 15 – Общая схема алгоритма синтеза изображения

2.4 Аффинные преобразования

В представленном алгоритме синтеза изображения первым этапом преобразования полигона перед его растеризацией является переход модели в мировое пространство. Такое действие осуществляется с помощью матриц аффинных преобразований [8]. В данном курсовом проекте над объектами возможно произвести следующие операции.

- Поворот вокруг координатных осей описывается углом α и осью вращения. Матрица поворота имеет вид:

- вокруг оси OX:

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\alpha & \sin\alpha & 0 \\ 0 & -\sin\alpha & \cos\alpha & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad (6)$$

- вокруг оси OY:

$$\begin{pmatrix} \cos\alpha & 0 & -\sin\alpha & 0 \\ 0 & 1 & 0 & 0 \\ \sin\alpha & 0 & \cos\alpha & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad (7)$$

- вокруг оси OZ:

$$\begin{pmatrix} \cos\alpha & \sin\alpha & 0 & 0 \\ -\sin\alpha & \cos\alpha & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad (8)$$

- Перенос в трехмерном пространстве задается значения вдоль координатных осей OX, OY, OZ — dx , dy , dz соответственно. Матрица переноса

имеет вид:

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ dx & dy & dz & 1 \end{pmatrix} \quad (9)$$

2.5 Приведение к пространству камеры

Для перемещения по сцене используется камера, задаваемая точкой положения в пространстве, пирамидой видимости и собственной системой координат, которая состоит из трех ортогональных векторов.

Обозначим:

- P — положение камеры;
- D — вектор взгляда;
- U — вектор вверх;
- R — вектор вправо.

Для перехода в пространство камеры выполняется в два этапа, указанные далее.

- 1) Перенос полигона в отрицательную стороны от камеры на расстояние P с помощью матрицы переноса [1]:

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ -Px & -Py & -Pz & 1 \end{pmatrix} \quad (10)$$

- 2) Преобразование полигона к системе координат камеры при помощи

матрицы поворота [1]:

$$\begin{pmatrix} Rx & Ux & Dx & 0 \\ Ry & Uy & Dy & 0 \\ Rz & Uz & Dz & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad (11)$$

Управление камеры производится с помощью изменения углом Эйлера. Обозначим α — угол поворота вокруг оси ОУ (тангаж) и β — угол поворота вокруг оси ОХ (рыскание). Тогда координаты вектора направления камеры можно вычислить по следующим формулам:

$$D_x = \cos(\alpha) \cdot \cos(\beta) \quad (12)$$

$$D_y = \sin(\alpha) \quad (13)$$

$$D_z = \cos(\alpha) \cdot \sin(\beta) \quad (14)$$

2.6 Перспективная проекция

После перехода в пространство камеры необходимо спроецировать полигон на картинную плоскость. В данном курсовом проекте используется перспективная проекция. Обозначим:

- $zoom_x$ — приближение по X;
- $zoom_y$ — приближение по Y;
- R — соотношение сторон экрана;
- F — расстояние от камеры до задней грани пирамиды видимости;
- N — расстояние от камеры до передней грани пирамиды видимости;
- γ — вертикальный угол обзора.

Увеличение объектов по координатам X и Y можно вычислить по формулам:

$$zoom_y = \frac{1}{tg(\gamma/2)} \quad (15)$$

$$zoom_x = \frac{zoom_y}{R} \quad (16)$$

Для перехода в пространство отсечения используется матрица перспективной проекции:

$$\begin{pmatrix} zoom_x & 0 & 0 & 0 \\ 0 & zoom_y & 0 & 0 \\ 0 & 0 & \frac{F+N}{F-N} & 1 \\ 0 & 0 & \frac{-2 \cdot F \cdot N}{F-N} & 0 \end{pmatrix} \quad (17)$$

2.7 Невидимые грани

С помощью отбрасывания нелицевых граней моделей при построении изображения можно существенно сократить временные затраты, так как не полигоны, невидимые по отношению к камере, растеризоваться не будут. Для определения видимости грани требуется использовать формулу:

$$(\vec{N}, \vec{V}) = \begin{cases} \geq 0, & \text{если грань невидима} \\ < 0, & \text{если грань видима} \end{cases}, \quad (18)$$

где $\vec{N}$ — вектор внешней нормали к грани модели, $\vec{V}$ — вектор от камеры до любой точки грани.

2.8 Пирамида видимости

Отсечение полигона по пирамиде видимости решает две задачи:

- 1) удаление полигонов, лежащих за камерой, но проецируемых на картинную плоскость в перевернутом виде;
- 2) сокращение времени на растеризацию полигонов за счет удаления их невидимых частей.

Для выполнения операции отсечения в данном курсовом проекте использовался алгоритм Сазерленда-Ходжмана [4]. Согласно рисунку 16 его идея состоит в том, чтобы последовательно отсекал полигон относительно каждой грани пирамиды видимости до тех пор, пока полигон не будет отсечен полностью либо пока все грани не будут пройдены.

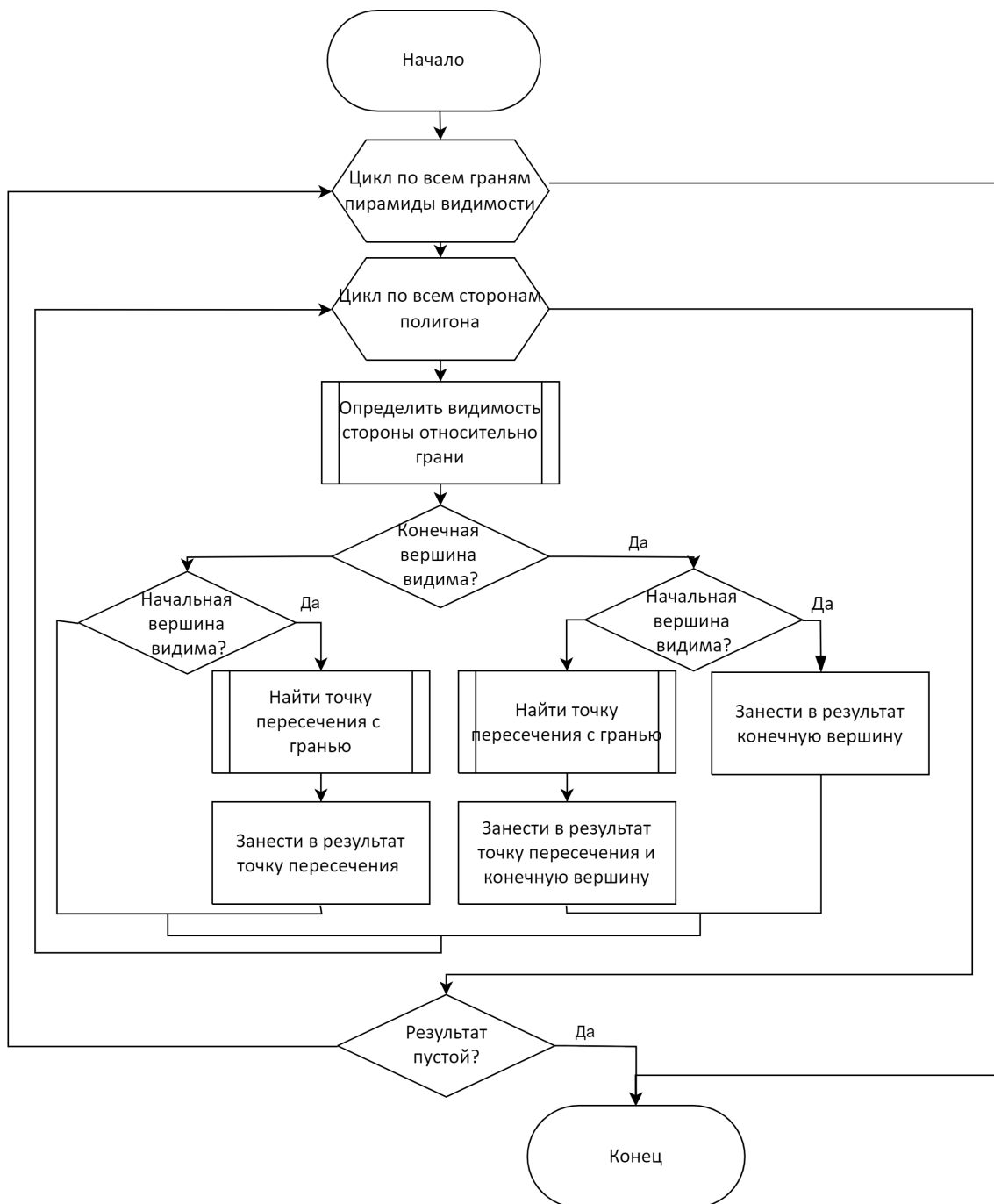


Рисунок 16 – Схема алгоритма Сазерленда-Ходжмана

2.9 Алгоритм Z-буфера

Для растеризации треугольного полигона сначала необходимо найти ограничивающий его прямоугольник, в котором этот полигон содержится. Это делается для того, чтобы не тратить время на растеризацию пикселей, не являющихся частью полигона. Затем для каждого пикселя ограничивающего прямоугольника находятся его барицентрические координаты относительно вершин полигона.

Пусть A, B, C – вершины грани, P – пиксель внутри ограничивающего прямоугольника. Площадь треугольника можно найти по следующей формуле:

$$square = (A_y - C_y) \cdot (B_x - C_x) + (B_y - C_y) \cdot (C_x - A_x) \quad (19)$$

Тогда барицентрические координаты пикселя равны:

$$\alpha = \frac{(P_y - C_y) \cdot (B_x - C_x) + (B_y - C_y) \cdot (C_x - P_x)}{square} \quad (20)$$

$$\beta = \frac{(P_y - A_y) \cdot (C_x - A_x) + (C_y - A_y) \cdot (A_x - P_x)}{square} \quad (21)$$

$$\gamma = 1 - \alpha - \beta \quad (22)$$

В случае, если хоть одна из барицентрических координат отрицательна, то пиксель лежит вне полигона. Если пиксель лежит внутри треугольника, то найти значение его глубины можно по следующей формуле:

$$z = \frac{1}{\frac{\alpha}{A_z} + \frac{\beta}{B_z} + \frac{\gamma}{C_z}} \quad (23)$$

Затем производится сравнение значения глубины точки со значением глубины из Z-буфера. Если глубина пикселя меньше, значит он лежит ближе

к камере и должен быть растеризован. Происходит вычисление интенсивности пикселя, его значение заносится в буфер кадра, а в Z-буфер заносится значение глубины пикселя. Полная схема алгоритма Z-буфера представлена на рисунке 17.

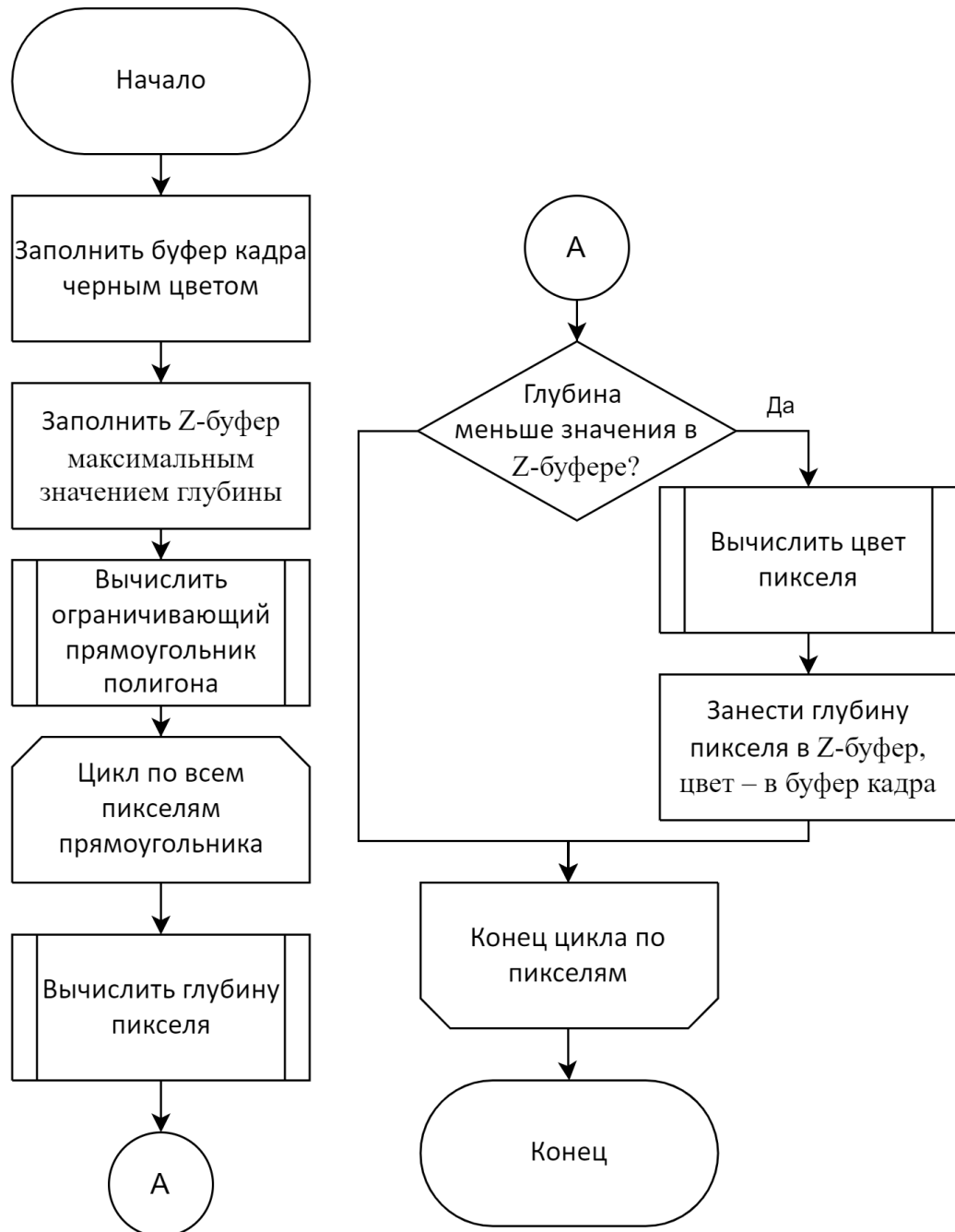


Рисунок 17 – Схема алгоритма Z-буфера

Вывод

Были описаны требования к программному обеспечению, структур данных, алгоритмы для построения сцены в пространстве изображения, изменения положения объекта в пространстве, построение камеры и ее проекций.

3 Технологическая часть

В данной части рассматривается выбор средств реализации, описывается структура классов программы и приводится интерфейс программного обеспечения.

3.1 Средства реализации

Для написания данного курсового проекта был выбран язык C++ [9]. Выбор данного языка программирования обусловлен следующим образом:

- поддерживает объектно-ориентированную модель разработки, что позволяет структурировать программу и дает возможность эффективного написания качественного программного обеспечения;
- позволяет эффективно использовать ресурсы системы благодаря широкому набору функций и классов из стандартной библиотеки;
- обладает высокими показателями вычислительной производительности, а так как требуется быстроедействие задач генерации реалистичных изображений, то язык C++ необходим.

В качестве среды разработки был использован Qt Creator [10]. Он обладает всем необходимым функционалом для написания, профилирования и отладки программ, а также создания графического пользовательского интерфейса. Данная среда поставляется с фреймворком Qt [11], который содержит в себе все необходимые средства, позволяющие работать непосредственно с пикселями изображения. Для упрощения сборки проекта программного обеспечения использовалась утилита qmake [12].

3.2 Структура программы

На рисунке 18 представлена схема разработанных классов.

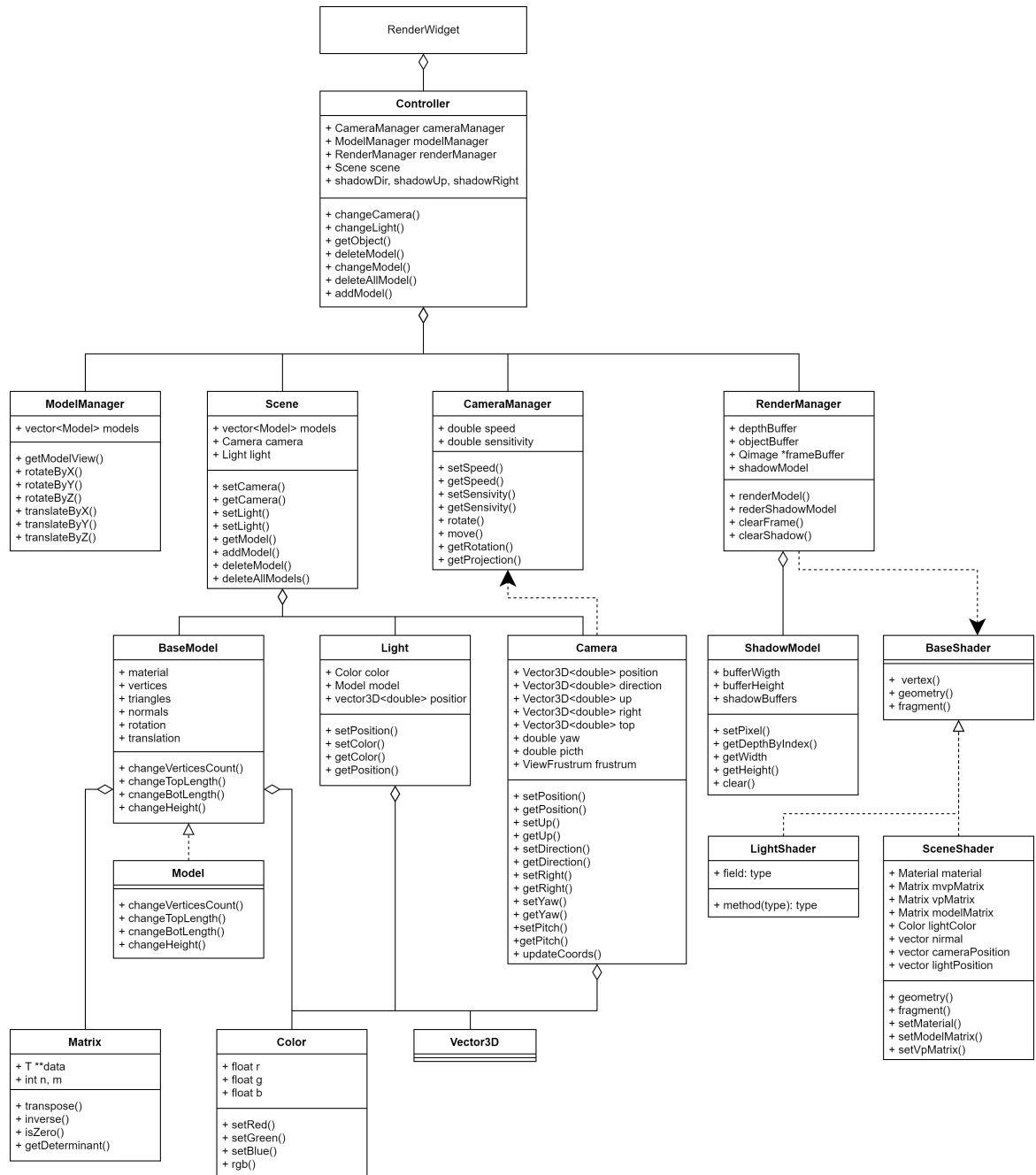


Рисунок 18 – Схема классов программы

Разработанная программа состоит из следующих классов.

- Базовые математические классы:
 - Matrix — класс матриц;
 - Vector3D — класс векторов трехмерного пространства;

- Vector4D — класс векторов в однородных координатах.
- Классы для работы с моделями
 - BaseModel — базовый класс моделей с возможностью перемещения по сцене и изменения спектральных характеристик;
 - Model — класс модели с параметрами радиус верхнего и нижнего оснований, высота и число боковых граней, которые можно меняться.
- Вспомогательные классы сцены
 - Camera — класс камеры с возможностью перемещения по сцене;
 - Light — класс источника освещения с возможностью перемещения по сцене и изменения цвета.
- Классы рейдеров
 - BaseShader — базовый класс шейдера для преобразования вершин и вычисления интенсивности пикселей;
 - SceneShader — класс шейдера для визуализации сцены;
 - LightShader — класс шейдера для визуализации источника освещения.
- Классы управления сценой
 - ModelManager — класс для осуществления аффинных преобразований;
 - CameraManager — класс для осуществления видовых и перспективных преобразований;
 - RenderManager — класс для визуализации сцены;
 - Scene — класс управления объектами сцены.
- Классы интерфейса
 - Controller — класс для взаимодействия управляющих классов с классами интерфейса;
 - RenderWidget — класс виджета сцены для вывода буфера кадра;
 - MainWindow — класс главного окна сцены.

3.3 Реализация алгоритмов

На листингах 1 – 5 приведена реализация алгоритма визуализации сцены по схеме алгоритма синтеза изображения, который показан на рисунке 15. В листинге 5 приведена многопоточная реализация алгоритм с Z-буфером.

Основной поток создает вспомогательные потоки и разбивает растеризацию полигона между ними, данную реализацию можно рассмотреть на листингах 3–4. В качестве аргументов потоку передается:

- `startCol` — номер строки буфера для начало работы;
- `endCol` — номер строки буфера для конца работы потока;
- `startRow` — номер столбца буфера для начало работы потока в строках $[startCol; endCol)$;
- `endRow` — номер столбца буфера для конца работы потока в строках $[startCol; endCol)$;

По аргументам будет определен объем работы, выполняемый конкретным вспомогательным потоком.

На листингах 6 – 9 представлена реализация алгоритма визуализации теней на сцене с использованием карты теней.

На листингах 12 – 10 представлены реализация поворота и перемещения модели. На листингах 14 – 15 представлены реализации движения и поворота камеры, вычисления перспективной проекции пространства камеры.

Листинг 1 – Реализация визуализации сцены

```
1 void Controller::renderScene()
2 {
3     Camera camera = scene.getCamera();
4     Light light = scene.getLight();
5     renderManager.clearFrame();
6     Matrix<double> rotation = cameraManager.getRotation(camera);
7     Matrix<double> vpMatrix = cameraManager.getLookAt(camera) *
        cameraManager.getProjection(camera);
8     Matrix<double> modelMatrix(4, 4);
9     SceneShader sceneShader;
10    Model model;
11    sceneShader.setCameraPosition(camera.getPosition());
12    sceneShader.setLightPosition(light.getPosition());
13    sceneShader.setVpMatrix(vpMatrix);
14    for (int i = 0; i < scene.countModels(); i++)
15    {
16        model = scene.getModel(i);
17        modelMatrix = modelManager.getModelView(model);
18        sceneShader.setModelMatrix(modelMatrix);
19        sceneShader.setMaterial(model.getMaterial());
20        sceneShader.setLightColor(light.getColor());
21        renderManager.renderModel(sceneShader, model, i);
22    }
23    LightShader lightShader;
24    lightShader.setCameraPosition(camera.getPosition());
25    lightShader.setVpMatrix(vpMatrix);
26    model = light.getModel();
27    modelMatrix = modelManager.getModelView(model);
28    lightShader.setModelMatrix(modelMatrix);
29    lightShader.setLightColor(light.getColor());
30    renderManager.renderModel(lightShader, model, -1);
31    renderManager.renderCoordinateSystem(rotation);
32 }
```

Листинг 2 – Реализация визуализации модели на сцене

```
1 void RenderManager::renderModel(BaseShader &shader, const
   BaseModel &model, const int &index)
2 {
3     char objectIndex = index;
4     std::vector<Vector3D<double>> triangle(3);
5     std::vector<Vector4D<double>> result(9);
6     int count;
7     for (int j = 0; j < model.countTriangles(); j++)
8     {
9         model.getTriangle(triangle, j);
10        count = shader.vertex(result, triangle,
11                               model.getNormal(j));
12        for (int i = 0; i < count - 2; i++)
13        {
14            shader.geometry({result[i+2], result[i+1],
15                             result[0]});
16            triangle[2] = result[0];
17            triangle[1] = result[i + 1];
18            triangle[0] = result[i + 2];
19            renderTriangle(triangle, objectIndex, shader);
20        }
21    }
22 }
```


Листинг 3 – Реализация визуализации полигона модели на сцене (Часть 1)

```
1 void RenderManager::renderTriangle(std::vector<Vector3D<double>>
   &triangle, const char &objectIndex, const BaseShader &shader)
2 {
3     for (auto &point: triangle)
4     {
5         viewPort(point, screenWidth, screenHeight);
6         point.setZ(1 / point.z());
7     }
8     QPoint leftCorner(std::numeric_limits<int>::max(),
        std::numeric_limits<int>::max());
9     QPoint rightCorner(0, 0);
10    for (const auto &point: triangle)
11    {
12        rightCorner.setX(std::max(rightCorner.x(),
            static_cast<int>(point.x())));
13        rightCorner.setY(std::max(rightCorner.y(),
            static_cast<int>(point.y())));
14        leftCorner.setX(std::min(leftCorner.x(),
            static_cast<int>(point.x())));
15        leftCorner.setY(std::min(leftCorner.y(),
            static_cast<int>(point.y())));
16    }
17    rightCorner.setX(std::min(rightCorner.x(), screenWidth - 1));
18    rightCorner.setY(std::min(rightCorner.y(), screenHeight - 1));
19    double square = (triangle[0].y() - triangle[2].y()) *
        (triangle[1].x() - triangle[2].x()) +
20        (triangle[1].y() - triangle[2].y()) *
        (triangle[2].x() - triangle[0].x());
21    std::vector<std::thread> threads(8);
22    ThreadParams params;
23    params.startRow = leftCorner.y();
24    params.endRow = rightCorner.y();
25    double start = leftCorner.x();
```

Листинг 4 – Реализация визуализации полигона модели на сцене (Часть 2)

```
1    double step = (rightCorner.x() - leftCorner.x()) / 8.0;
2    for (auto &thread: threads)
3    {
4        params.startCol = round(start);
5        params.endCol = round(start + step);
6        thread = std::thread(&RenderManager::renderFrameBuffer,
7                               this, params, std::cref(triangle),
8                               std::cref(objectIndex), std::cref(shader),
9                               std::cref(square));
7        start += step;
8    }
9    for (auto &thread: threads)
10   {
11       thread.join();
12   }
13 }
```

Листинг 5 – Реализация алгоритма с z-буфером

```
1 void RenderManager::renderFrameBuffer(ThreadParams params, const
  std::vector<Vector3D<double>> &triangle, const char
  &objectIndex, const BaseShader &shader, const double &square)
2 {
3     Vector3D<double> barCoords;
4     double z;
5     Color color;
6     for (int i = params.startCol; i <= params.endCol; i++)
7     {
8         for (int j = params.startRow; j <= params.endRow; j++)
9         {
10             barycentric(barCoords, triangle, QPoint(i, j),
              square);
11             if (barCoords.x() >= -EPS && barCoords.y() >= -EPS &&
              barCoords.z() >= -EPS)
12             {
13                 z = 1 / (barCoords.x() * triangle[0].z() +
                  barCoords.y() * triangle[1].z() +
                  barCoords.z() * triangle[2].z());
14                 if (z <= depthBuffer[j][i])
15                 {
16                     depthBuffer[j][i] = z;
17                     objectsBuffer[j][i] = objectIndex;
18                     color = shader.fragment(barCoords,
                      shadowModel);
19                     frameBuffer->setPixel(i, j, color.rgb());
20                 }
21             }
22         }
23     }
24 }
```

Листинг 6 – Реализация визуализации теней на сцене

```
1 void Controller::renderShadow()
2 {
3     Camera camera;
4     Light light = scene.getLight();
5     camera.setViewFrustrum({90, 0.1, 1000, 1});
6     renderManager.clearShadow();
7     Matrix<double> projection =
8         cameraManager.getProjection(camera);
9     Matrix<double> modelMatrix(4, 4);
10    Matrix<double> vpMatrix(4, 4);
11    SceneShader shader;
12    shader.setCameraPosition(light.getPosition());
13    shader.setLightPosition(light.getPosition());
14    camera.setPosition(light.getPosition());
15    for (int j = 0; j < 6; j++)
16    {
17        camera.setDirection(shadowDir[j]);
18        camera.setUp(shadowUp[j]);
19        camera.setRight(shadowRight[j]);
20        vpMatrix = cameraManager.getLookAt(camera) * projection;
21        shader.setVpMatrix(vpMatrix);
22        for (int i = 0; i < scene.countModels(); i++)
23        {
24            Model model = scene.getModel(i);
25            modelMatrix = modelManager.getModelView(model);
26            shader.setModelMatrix(modelMatrix);
27            renderManager.renderShadowModel(shader, model, j);
28        }
29    }
```

Листинг 7 – Реализация визуализации теней для модели на сцене

```
1 void RenderManager::renderShadowModel(SceneShader &shader, const
   BaseModel &model, const int &bufferIndex)
2 {
3     std::vector<Vector3D<double>> triangle(3);
4     std::vector<Vector4D<double>> result(9);
5     int count;
6     for (int j = 0; j < model.countTriangles(); j++)
7     {
8         model.getTriangle(triangle, j);
9         count = shader.vertex(result, triangle,
10                                model.getNormal(j));
11         for (int i = 0; i < count - 2; i++)
12         {
13             shader.geometry({result[i+2], result[i+1],
14                               result[0]});
15             triangle[2] = result[0];
16             triangle[1] = result[i + 1];
17             triangle[0] = result[i + 2];
18             renderShadowTriangle(triangle, bufferIndex, shader);
19         }
20     }
21 }
```

Листинг 8 – Реализация визуализации теней для полигона модели на сцене с использованием карты теней (Часть 1)

```
1 void
  RenderManager::renderShadowTriangle(std::vector<Vector3D<double>
  > &triangle, const int &bufferIndex, const SceneShader &shader)
2 {
3     for (auto &point: triangle)
4     {
5         viewPort(point, shadowModel.getWidth(),
6                 shadowModel.getHeight());
7         point.setZ(1 / point.z());
8         QPoint leftCorner(std::numeric_limits<int>::max(),
9                           std::numeric_limits<int>::max());
10        QPoint rightCorner(0, 0);
11        for (const auto &point: triangle)
12        {
13            rightCorner.setX(std::max(rightCorner.x(),
14                                     static_cast<int>(point.x())));
15            rightCorner.setY(std::max(rightCorner.y(),
16                                     static_cast<int>(point.y())));
17            leftCorner.setX(std::min(leftCorner.x(),
18                                    static_cast<int>(point.x())));
19            leftCorner.setY(std::min(leftCorner.y(),
20                                    static_cast<int>(point.y())));
21        }
22        rightCorner.setX(std::min(rightCorner.x(),
23                                  shadowModel.getWidth() - 1));
24        rightCorner.setY(std::min(rightCorner.y(),
25                                  shadowModel.getHeight() - 1));
26        double square = (triangle[0].y() - triangle[2].y()) *
27                        (triangle[1].x() - triangle[2].x()) +
28                        (triangle[1].y() - triangle[2].y()) *
29                        (triangle[2].x() - triangle[0].x());
```

Листинг 9 – Реализация визуализации теней для полигона модели на сцене с использованием карты теней (Часть 2)

```
1   Vector3D<double> barCoords;
2   double z;
3   for (int i = leftCorner.x(); i <= rightCorner.x(); i++)
4   {
5       for (int j = leftCorner.y(); j <= rightCorner.y(); j++)
6       {
7           barycentric(barCoords, triangle, QPoint(i, j),
8                       square);
9           if (barCoords.x() >= -EPS && barCoords.y() >= -EPS &&
10              barCoords.z() >= -EPS)
11           {
12               z = shader.countShadowDepth(barCoords);
13               if (z <= shadowModel.getDepthByIndex(bufferIndex,
14              i, j))
15               {
16                   shadowModel.setPixel(bufferIndex, i, j, z);
17               }
18           }
19       }
20   }
```

Листинг 10 – Реализация аффинного преобразования — поворота модели
(Часть 1)

```
1 void ModelManager::rotateByX(BaseModel &model, const double
   &angle) const
2 {
3     double rad = angle * M_PI / 180;
4     Matrix<double> rotateX(4, 4);
5     rotateX.makeIdentity();
6     rotateX[1][1] = cos(rad);
7     rotateX[1][2] = sin(rad);
8     rotateX[2][1] = -rotateX[1][2];
9     rotateX[2][2] = rotateX[1][1];
10    model.setRotation(model.getRotation() * rotateX);
11 }
12
13 void ModelManager::rotateByY(BaseModel &model, const double
   &angle) const
14 {
15     double rad = angle * M_PI / 180;
16     Matrix<double> rotateY(4, 4);
17     rotateY.makeIdentity();
18     rotateY[0][0] = cos(rad);
19     rotateY[0][2] = -sin(rad);
20     rotateY[2][0] = -rotateY[0][2];
21     rotateY[2][2] = rotateY[0][0];
22     model.setRotation(model.getRotation() * rotateY);
23 }
24
25 void ModelManager::rotateByZ(BaseModel &model, const double
   &angle) const
26 {
27     double rad = angle * M_PI / 180;
```


Листинг 11 – Реализация аффинного преобразования — поворота модели
(Часть 2)

```
1   Matrix<double> rotateZ(4, 4);
2   rotateZ.makeIdentity();
3   rotateZ[0][0] = cos(rad);
4   rotateZ[0][1] = sin(rad);
5   rotateZ[1][0] = -rotateZ[0][1];
6   rotateZ[1][1] = rotateZ[0][0];
7   model.setRotation(model.getRotation() * rotateZ);
8 }
```

Листинг 12 – Реализация аффинного преобразования — перемещение модели

```
1 inline void translateByX(BaseModel &model, const double
   &translation) const {
2     model.setTranslation(model.getTranslation() +
       Vector3D<double>(translation, 0, 0));
3 }
4 inline void translateByY(BaseModel &model, const double
   &translation) const {
5     model.setTranslation(model.getTranslation() +
       Vector3D<double>(0, translation, 0));
6 }
7 inline void translateByZ(BaseModel &model, const double
   &translation) const {
8     model.setTranslation(model.getTranslation() +
       Vector3D<double>(0, 0, translation));
9 }
```

Листинг 13 – Реализация поворота камеры

```
1 void CameraManager::rotate(Camera &camera, const CameraRotation
   &rotation)
2 {
3     double yaw = camera.getYaw();
4     double pitch = camera.getPitch();
5     yaw += rotation.dYaw * sensitivity_;
6     pitch += rotation.dPitch * sensitivity_;
7     if (pitch >= 90)
8     {
9         pitch = 90;
10    }
11    else if (pitch <= -90)
12    {
13        pitch = -90;
14    }
15    if (yaw >= 180)
16    {
17        yaw = -180;
18    }
19    else if (yaw <= -180)
20    {
21        yaw = 180;
22    }
23    camera.setYaw(yaw);
24    camera.setPitch(pitch);
25    camera.updateCoords();
26 }
```

Листинг 14 – Реализация движения камеры

```

1 void CameraManager::move(Camera &camera, const CameraMovement
    &movement)
2 {
3     Vector3D<double> position(camera.getPosition());
4     Vector3D<double> direction(camera.getDirection());
5     Vector3D<double> right(camera.getRight());
6     if (movement.forward)
7         position += direction * speed_;
8     if (movement.backward)
9         position -= direction * speed_;
10    if (movement.left)
11        position -= right * speed_;
12    if (movement.right)
13        position += right * speed_;
14    camera.setPosition(position);
15 }

```

Листинг 15 – Реализация алгоритма вычисления перспективной проекции

```

1 Matrix<double> CameraManager::getProjection(const Camera &camera)
    const {
2     ViewFrustrum frustrum = camera.getViewFrustrum();
3     Matrix<double> projection(4,4);
4     projection[1][1] = 1 / tan(radians(frustrum.field_of_view /
        2));
5     projection[0][0] = projection[1][1] / frustrum.aspect_ratio;
6     projection[2][2] = (frustrum.far_plane + frustrum.near_plane)
        / (frustrum.far_plane - frustrum.near_plane);
7     projection[3][2] = -2 * frustrum.near_plane *
        frustrum.far_plane / (frustrum.far_plane -
        frustrum.near_plane);
8     projection[2][3] = 1;
9     return projection;
10 }

```

3.4 Интерфейс программного обеспечения

При запуске программы на виджете сцены отображается источник освещения и система координат в левом нижнем углу. Источник освещения отображается как пирамида цвета источника. Для операций над объектами в левой части интерфейса определены разделы (создания и изменения объекта) и для управления камерой или освещением используются разделы в правой части экрана (рисунок 19).

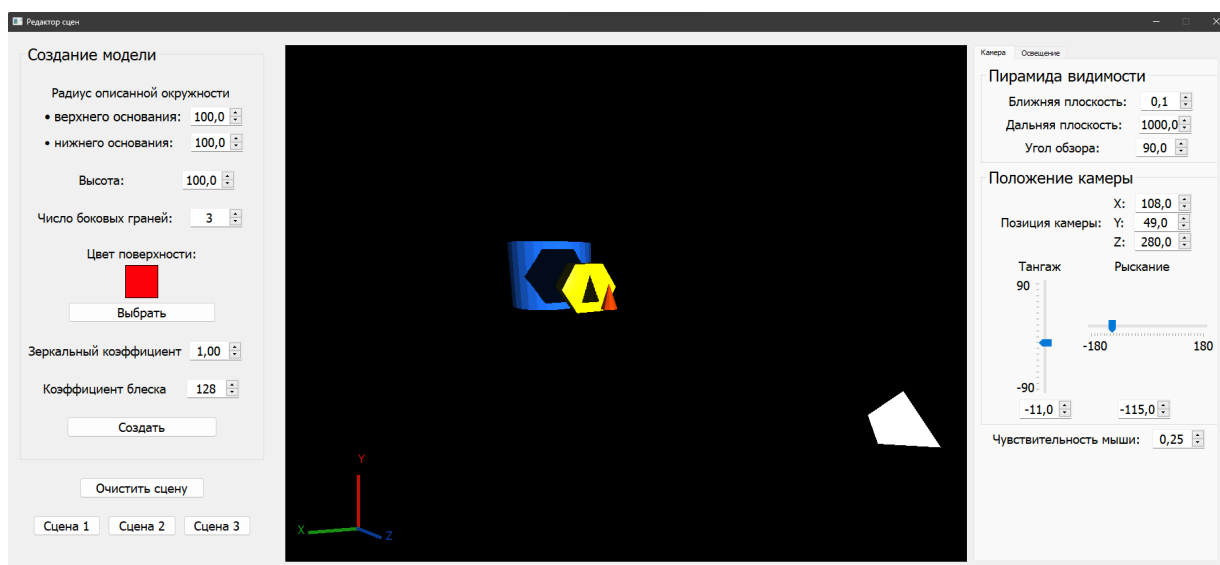


Рисунок 19 – GUI

Для создания модели пользователю необходимо ввести в разделе «Создание модели» ее геометрические и спектральные характеристики: радиусы верхнего и нижнего оснований, высоту, число боковых сторон, цвет поверхности, коэффициент блеска и зеркальный коэффициент, раздел показан на рисунке 20.

Создание модели

Радиус описанной окружности

- верхнего основания: 100,0
- нижнего основания: 100,0

Высота: 100,0

Число боковых граней: 3

Цвет поверхности:



Выбрать

Зеркальный коэффициент 1,00

Коэффициент блеска 128

Создать

Рисунок 20 – Раздел создание модели

При нажатии правой кнопкой мыши по модели на виджете сцены в разделе «Создание модели»s отобразится секция с редактированием модели, раздел показан на рисунке 21. Для возврата в секцию создания необходимо нажать правой кнопкой мыши по пустой части виджета сцены.

Для перемещения по сцене используются клавиши: W – вперед, S – назад, A – влево, D – вправо. Для поворота камеры необходимо зажать левую кнопку мыши и переместить мышь.

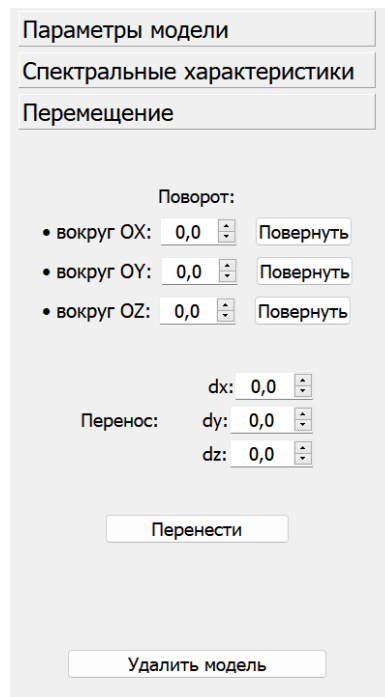


Рисунок 21 – Раздел редактирование модели

В разделе «Камер» находятся следующие параметры: расстояния до ближней и дальней плоскостей пирамиды видимости, угол обзора, позиция камеры в пространстве, углы поворота, скорость перемещения и чувствительность мыши (рисунок 22).

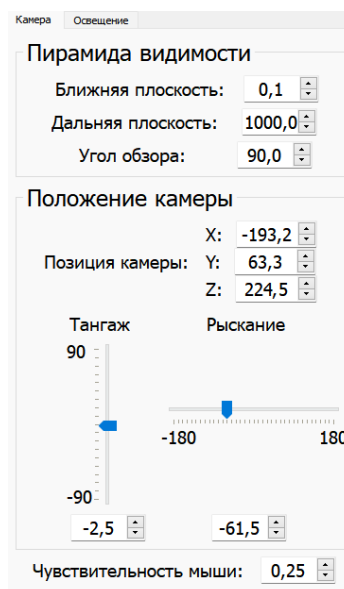


Рисунок 22 – Раздел камеры

Раздел «Освещение» содержит положение и цвет источника освещения (рисунок 23). При выборе цвета появляется диалоговое окно с палитрой цве-

тов (рисунок 24).

Камера Освещение

Позиция источника

X: -100,0

Y: 110,0

Z: 170,0

Цвет источника

Выбрать

Рисунок 23 – Раздел освещения

Выберите цвет

Basic colors

Pick Screen Color

Custom colors

Add to Custom Colors

Hue: 0 Red: 255

Sat: 0 Green: 255

Val: 255 Blue: 255

HTML: #ffffff

OK Cancel

Рисунок 24 – Окно выбора цвета

На рисунке 25 приведен пример работы программы.

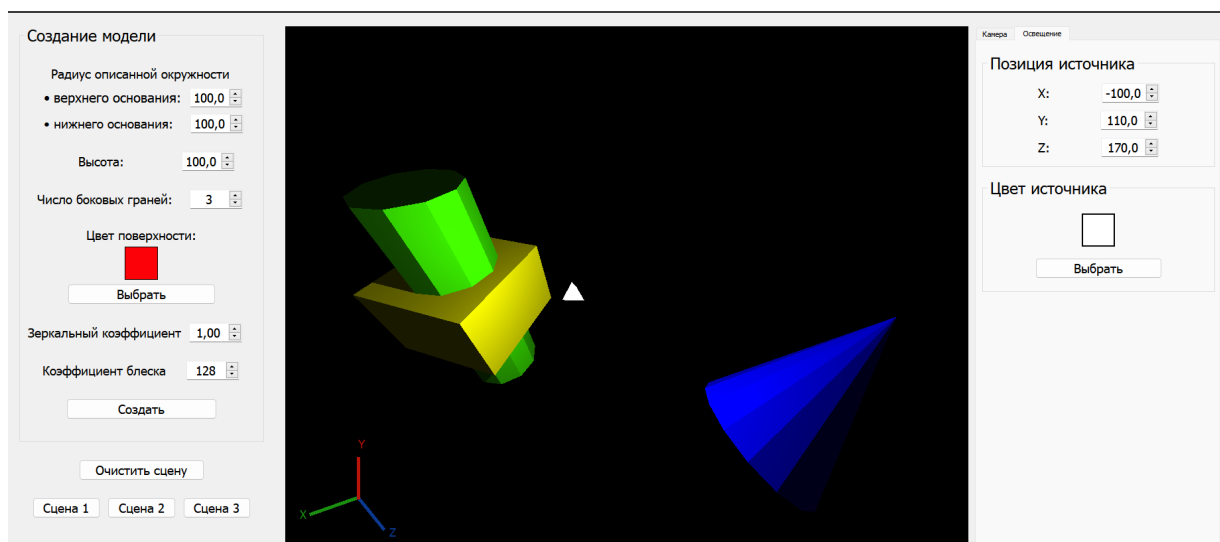


Рисунок 25 – Тестовая сцена

Вывод

В данном разделе были выбраны средства реализации, описаны структуры классов программы, описаны модули, а также рассмотрен интерфейс программы

4 Исследовательская часть

В данном разделе приведены технические характеристики устройства, на котором проводилось измерение времени работы программного обеспечения, а также результаты замеров времени.

4.1 Постановка эксперимента

Целью эксперимента является провести анализ скорости работы алгоритма генерации изображения с использованием алгоритма Z-буфером.

4.1.1 Технические характеристики

Технические характеристики устройства, на котором выполнялось тестирование.

- Процессор: Intel(R) Core(TM) i5-10300H CPU 2.50 ГГц [13].
- Количество ядра: 4 физических и 8 логических ядер.
- Оперативная память: 16 ГБайт.
- Операционная система: Windows 11 Pro 64-разрядная система [14].

Тестирование проводилось на стационарном компьютере. Во время тестирования устройство было нагружено только встроенными приложениями окружения, а также непосредственно системой тестирования.

4.1.2 Результаты эксперимента

Для исследования зависимости времени тренинга изображения от числа объектов на сцене, использовались объекты с фиксированным числом граней,

каждый объект имел освещенную и затененную части. Количество объектов менялось на сцене от 100 до 1000 с шагом 100, были рассмотрены случаи для 3-х, 6-ти и 9-тигранных объектов. Результаты проведенного исследования представлены.

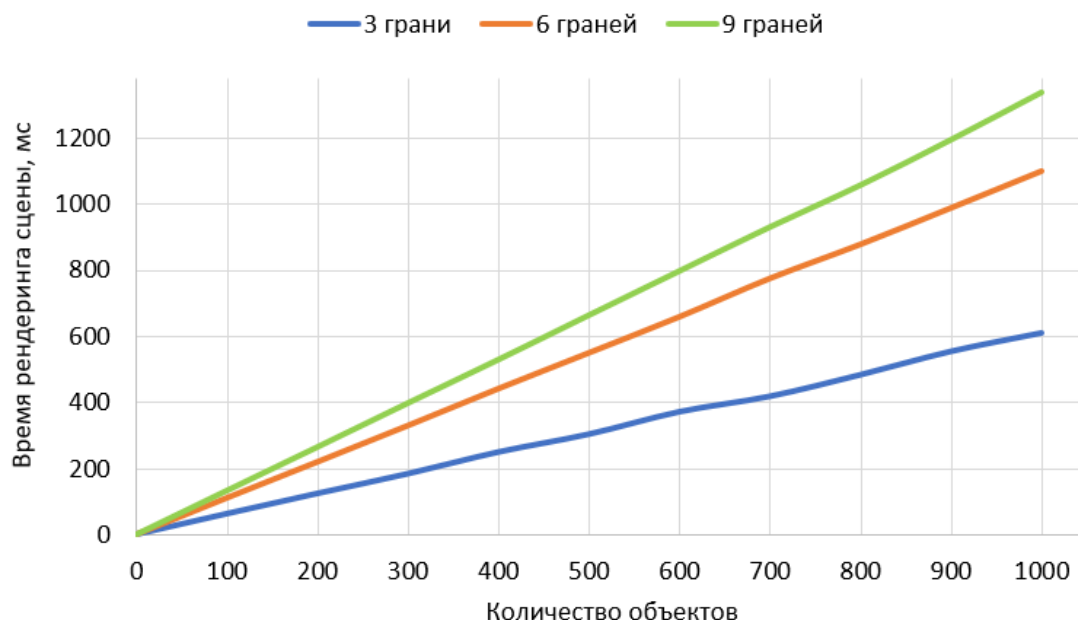


Рисунок 26 – График зависимости времени отрисовки от числа объектов

Как видно из графика, время визуализации сцены зависит от количества объектов линейно, независимо от числа их боковых граней.

Следующим этапом исследования разработанной программы является исследование зависимости времени построения сцены от числа боковых граней объектов при фиксированном количестве объектов. В ходе эксперимента количество боковых граней менялось от 50 до 500 с шагом 50, при этом были рассмотрены случаи для 2-х объектов с равным числом граней и для 2-х объектов, у одного из которых число граней менялось, а у другого было фиксировано.

Из проведенного эксперимента можно сделать вывод, что время визуализации сцены линейно зависит от числа боковых граней объектов, при этом для 2-х объектов время растет вдвое быстрее, если количество граней у обоих увеличивается одновременно, чем в случае фиксированного количества

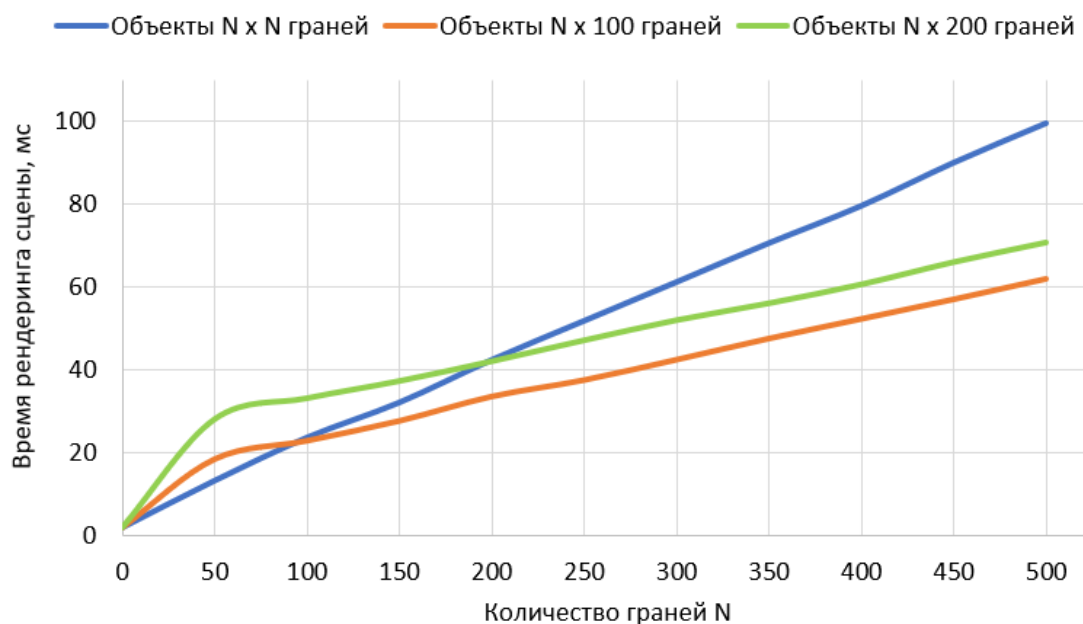


Рисунок 27 – График зависимости времени отрисовки от числа боковых граней

граней у одного из них. В случаях фиксированного количества граней время растёт одинаково, постоянная разница во времени объясняется разным числом граней у фиксированного объекта.

Вывод

В данном разделе приведены результаты работы программного обеспечения и проведен эксперимент с использованием библиотеки `time`, функции которой использовались для определения эффективности работы программы по времени.

Результаты эксперимента совпали с ожидаемыми, так как в ходе эксперимента было установлено, что время работы увеличивается с увеличением количества объектов на сцене и количеством ребер у объекта.

ЗАКЛЮЧЕНИЕ

Целью данного курсового проекта была достигнута, то есть был разработан программный продукт, позволяющий создавать и редактировать композиции из трехмерных графических многогранников. Также предоставляет широкие возможности настройки геометрических и спектральных характеристик объектов, положения камеры и положения и цвета источника освещения. Были добавлены возможности изменения ориентации и положения камеры с помощью манипуляторов клавиатура и мышь.

Для достижение цели были выполнены следующие задачи:

- проведен анализ существующих алгоритмов компьютерной графики, использующие для создание реалистичной модели и трехмерной сцены;
- выбраны наиболее подходящих алгоритмов (алгоритмы удаления невидимых линий, методы закраски, модели освещения, алгоритмы создания динамических теней) для решения поставленной задачи ;
- спроектированы архитектуры и графического интерфейса программы;
- выбраны средства реализации программного обеспечения;
- разработаны ПО и реализация выбранных алгоритмов и структур данных;
- проведены замеры временных характеристик разработанного программного обеспечения.

Данный программный продукт может быть использован для демонстрации спектральной и диффузной отражающей способностей различных материалов, а также для визуализации падения теней при различных условиях освещенности.

Быстродействие низкоуровневых частей программы, отвечающих за rasterization полигонов, может быть улучшено за счет замены их программной реализации на поддерживаемую аппаратно из интерфейсов графических движков, например, OpenGL или DirectX.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1 Placing a Camera: the LookAt Function [Электронный ресурс]. — Режим доступа: <https://www.scratchapixel.com/lessons/mathematics-physics-for-computer-graphics/lookat-function/framing-lookat-function.html> (дата обращения: 20.09.2022).
- 2 Т.О. Перемилина. Компьютерная графика : учебное пособие. — Томск: Эль Контент, 2012. — С. 114.
- 3 Юрьевич Аксенов Алексей. Модели и методы обработки и представления сложных пространственных объектов. — СПИИРАН, 2015. — С. 110.
- 4 Д. Роджерс. Алгоритмические основы машинной графики: Пер. с англ. — СПб: БХВ-Петербург, 1989. — С. 512.
- 5 Шикин Е. В. Боресков А. В. Компьютерная графика. Динамика, реалистические изображения. — Москва: ДИАЛОГ-МИФИ, 1996. — С. 288.
- 6 Метод прямой и обратной трассировки [Электронный ресурс]. — Режим доступа: <https://en.cppreference.com/w/c/language> (дата обращения: 14.09.2022).
- 7 Простые модели освещения [Электронный ресурс]. — Режим доступа: <https://grafika.me/node/344> (дата обращения: 24.09.2022).
- 8 Н. Порев В. Компьютерная графика. — СПб.: БХВ-Петербург, 2002. — С. 429.
- 9 Документация по языку C++ [Электронный ресурс]. — Режим доступа: <https://learn.microsoft.com/ru-ru/cpp/cpp/?view=msvc-160> (дата обращения: 30.10.2022).

- 10 Qt Creator Manual [Электронный ресурс]. — Режим доступа:
<https://doc.qt.io/qtcreator> (дата обращения: 30.10.2022).
- 11 All Qt Documentation [Электронный ресурс]. — Режим доступа:
<https://doc.qt.io/all-topics.html> (дата обращения: 30.10.2022).
- 12 qmake Manual [Электронный ресурс]. — Режим доступа:
<https://doc.qt.io/qt-6/qmake-manual.html> (дата обращения:
30.10.2022).
- 13 Intel [Электронный ресурс]. — Режим доступа:
[https://ark.intel.com/content/www/ru/ru/ark/products/201839/
intel-core-i510300h-processor-8m-cache-up-to-4-50-ghz.html](https://ark.intel.com/content/www/ru/ru/ark/products/201839/intel-core-i510300h-processor-8m-cache-up-to-4-50-ghz.html)
(дата обращения: 25.09.2022).
- 14 Windows 11 Pro 64-bit [Электронный ресурс]. — Режим доступа:
<https://www.microsoft.com/ru-ru/software-download/windows10>
(дата обращения: 25.09.2022).

ПРИЛОЖЕНИЕ А

Презентация к курсовой работе

Презентация содержит 14 слайдов.